



Datenbanken und MariaDB Vom Fundament zur Expertise

Architektur, Modellierung, SQL, Indexierung, Query-Optimierung, Server-Tuning,
Hochverfügbarkeit, Backup und Betrieb

ERSTELLT FÜR

Timo Müller, Vorstand

STAND

06.09.2026

HINWEIS

Dieses Kompendium ist ein Lern- und Nachschlagewerk. Alle Parameterwerte sind Ausgangspunkte, die am eigenen System gemessen und verifiziert werden müssen. Versionsabhängige Angaben beziehen sich auf MariaDB 10.6 bis 11.x und sind gegen die jeweils eingesetzte Version zu prüfen.

Inhalt und Lernpfad

Das Dokument ist in vier Stufen aufgebaut. Jedes Kapitel trägt eine Stufenmarkierung. Wer bereits SQL beherrscht, beginnt bei Stufe 2. Wer produktive Systeme verantwortet, arbeitet Stufe 3 und 4 vollständig durch, da dort die Fehler entstehen, die Geld kosten.

1	Grundlagen: Was eine Datenbank leistet, relationales Modell, ACID, Systemlandschaft	STUFE 1
2	Datenmodellierung: Entitäten, Schlüssel, Normalisierung, Datentypen, Denormalisierung	STUFE 1
3	SQL von Grund auf: DDL, DML, Joins, Aggregation, Subqueries, CTEs, Window Functions	STUFE 1 BIS 2
4	MariaDB-Architektur: Server, Storage Engines, InnoDB im Detail, Speicher- und Logstrukturen	STUFE 2
5	Transaktionen, Isolation und Sperren: MVCC, Isolationsstufen, Deadlocks, Gap Locks	STUFE 2 BIS 3
6	Indexierung: B+-Baum, Clustered Index, zusammengesetzte Indizes, Covering, Anti-Muster	STUFE 2 BIS 3
7	Query-Optimierung: EXPLAIN, ANALYZE, Optimizer, Join-Strategien, typische Fehler, Hints	STUFE 3
8	Server-Tuning: die Parameter, die zählen, Speicher, I/O, Verbindungen, Betriebssystem	STUFE 3
9	Monitoring und Diagnose: Slow Log, Performance Schema, Statusvariablen, Werkzeuge	STUFE 3
10	Skalierung und Hochverfügbarkeit: Replikation, GTID, Galera, MaxScale, Partitionierung, Sharding	STUFE 3 BIS 4
11	Backup und Wiederherstellung: mariadb-dump, Mariabackup, Point-in-Time-Recovery, Teststrategie	STUFE 3
12	Sicherheit und Compliance: Rechte, Rollen, Verschlüsselung, Audit, SQL-Injection, DSGVO	STUFE 3
13	Betrieb und Schema-Evolution: Online-DDL, Migrationen, Upgrades, Wartung	STUFE 3 BIS 4
14	Fortgeschrittene Funktionen: JSON, System-Versioned Tables, Sequences, Prozeduren, Volltext, Vektoren	STUFE 4
15	Einordnung: MariaDB, MySQL, PostgreSQL und NoSQL im Vergleich	STUFE 4
16	Lernplan, Übungsaufgaben, Checklisten, Glossar	ALLE

WIE DIESES DOKUMENT ZU LESEN IST

Datenbankexpertise entsteht nicht durch Lesen, sondern durch Messen. Jeder Abschnitt zu Performance enthält Befehle, mit denen Sie das Verhalten am eigenen System sichtbar machen. Richten Sie sich eine Testinstanz ein (Docker reicht), laden Sie eine Tabelle mit einigen Millionen Zeilen und probieren Sie jede Aussage dieses Dokuments aus. Ein Experte unterscheidet sich vom Fortgeschrittenen dadurch, dass er weiß, wo die Faustregeln nicht gelten.

1. Grundlagen STUFE 1

1.1 Was eine Datenbank eigentlich leistet

Eine Datenbank ist mehr als ein Ort, an dem Daten liegen. Ein Datenbankmanagementsystem (DBMS) löst vier Probleme, die jede Anwendung sonst selbst lösen müsste: dauerhafte Speicherung mit garantierter Konsistenz auch bei Absturz, gleichzeitiger Zugriff vieler Nutzer ohne gegenseitige Zerstörung von Daten, effizientes Auffinden von Datensätzen in großen Bestände und eine deklarative Abfragesprache, mit der man beschreibt, was man will, nicht wie es zu finden ist.

Der letzte Punkt ist der entscheidende Denkwechsel. In SQL formulieren Sie das Ergebnis. Der Optimizer des Servers entscheidet über den Weg dorthin. Wer Datenbanken beherrschen will, muss deshalb verstehen, wie der Optimizer denkt, um ihm die richtigen Voraussetzungen (Indizes, Statistiken, Datentypen, Abfragestruktur) zu schaffen.

1.2 Das relationale Modell

Das relationale Modell (Edgar F. Codd, 1970) beschreibt Daten als Relationen, in der Praxis Tabellen. Eine Tabelle hat einen festen Satz benannter Spalten mit Datentypen und eine Menge von Zeilen. Die Reihenfolge der Zeilen ist bedeutungslos. Beziehungen zwischen Tabellen entstehen ausschließlich über Werte, nicht über Zeiger: Eine Rechnung verweist auf einen Kunden, indem sie dessen Kundennummer als Fremdschlüssel trägt.

BEGRIFF	BEDEUTUNG	BEISPIEL
Relation / Tabelle	Benannte Menge gleich strukturierter Datensätze	mietvertrag
Attribut / Spalte	Benanntes Merkmal mit Datentyp	kaltmiete DECIMAL(10,2)
Tupel / Zeile	Ein konkreter Datensatz	Vertrag 4711
Primärschlüssel	Eindeutige, nicht leere Kennung einer Zeile	vertrag_id
Fremdschlüssel	Verweis auf den Primärschlüssel einer anderen Tabelle	einheit_id in mietvertrag
Constraint	Regel, die der Server erzwingt	CHECK (kaltmiete >= 0)
Schema	Gesamtheit der Tabellendefinitionen (in MariaDB gleichbedeutend mit Datenbank)	CREATE DATABASE hv

1.3 ACID: Das Versprechen einer Transaktion

Eine Transaktion bündelt mehrere Schritte zu einer unteilbaren Einheit. Das klassische Beispiel ist eine Überweisung: Abbuchung und Gutschrift müssen beide oder gar nicht erfolgen. ACID beschreibt die vier Garantien.

EIGENSCHAFT	GARANTIE	TECHNISCHE UMSETZUNG IN INNODB
Atomicity	Alles oder nichts	Undo Log erlaubt vollständiges Zurückrollen

Consistency	Alle Regeln (Constraints, Fremdschlüssel) gelten vor und nach der Transaktion	Prüfung bei jedem Statement, Abbruch bei Verletzung
Isolation	Gleichzeitige Transaktionen beeinflussen sich nicht unzulässig	MVCC plus Sperren, konfigurierbar über Isolationsstufen
Durability	Bestätigte Änderungen überleben Abstürze	Redo Log wird vor dem Commit auf die Platte geschrieben (Write-Ahead Logging)

Merken Sie sich: Durability kostet I/O. Ein Großteil des Write-Tunings besteht darin zu entscheiden, wie viel Durability man wirklich braucht und wie viel man gegen Durchsatz tauscht (siehe Kapitel 8, `innodb_flush_log_at_trx_commit`).

1.4 Systemlandschaft: Wo MariaDB steht

MariaDB entstand 2009 als Abspaltung von MySQL, nachdem Oracle Sun Microsystems und damit MySQL übernommen hatte. Gründer war Michael Widenius, der ursprüngliche Autor von MySQL. MariaDB ist protokoll- und weitgehend syntaxkompatibel mit MySQL, hat sich aber seit Version 10.x eigenständig weiterentwickelt: eigene Storage Engines, System-Versioned Tables, Sequences, ein anderer Optimizer, Oracle-Kompatibilitätsmodus, eigenes Release-Modell.

KATEGORIE	VERTRETER	STÄRKE	TYPISCHER EINSATZ
Relational, zeilenorientiert (OLTP)	MariaDB, MySQL, PostgreSQL, Oracle, SQL Server	Transaktionen, Integrität, Punktzugriffe	Verwaltungssysteme, Buchhaltung, Webanwendungen
Relational, spaltenorientiert (OLAP)	MariaDB ColumnStore, ClickHouse, Snowflake	Aggregation über Milliarden Zeilen	Reporting, Data Warehouse
Dokumentenorientiert	MongoDB, Couchbase	Flexible Schemata, horizontale Skalierung	Produktkataloge, Ereignisdaten
Key-Value / Cache	Redis, Valkey, Memcached	Mikrosekunden-Latenz	Sessions, Caches, Zähler
Graph	Neo4j	Beziehungsabfragen beliebiger Tiefe	Netzwerke, Betrugserkennung
Zeitreihen	TimescaleDB, InfluxDB	Zeitfensteraggregation, Kompression	Sensorik, Monitoring

KAUFMÄNNISCHE EINORDNUNG

Für die überwiegende Zahl betrieblicher Anwendungen (Verwaltung, Abrechnung, CRM, ERP) ist ein relationales OLTP-System die richtige Wahl. NoSQL-Systeme lösen spezifische Skalierungs- oder Flexibilitätsprobleme und bringen dafür Verzicht auf Joins, Transaktionen über mehrere Datensätze oder starke Konsistenz mit. Wer keine dieser Probleme hat, kauft mit NoSQL Komplexität ohne Gegenwert.

1.5 OLTP und OLAP

OLTP (Online Transaction Processing) bedeutet viele kleine, kurze Transaktionen: eine Zeile lesen, drei Zeilen ändern, Commit. OLAP (Online Analytical Processing) bedeutet wenige, große Leseabfragen über viele Zeilen:

Summe aller Mieteinnahmen je Objekt je Monat der letzten fünf Jahre. Beide Muster stellen gegensätzliche Anforderungen an Speicherlayout, Indizes und Caching. Eine der häufigsten Ursachen für Performanceprobleme ist, Reporting-Abfragen ungebremst gegen die Produktivdatenbank laufen zu lassen. Die Standardlösung ist ein Replikat für Auswertungen (Kapitel 10).

1.6 CAP und die Grenzen verteilter Systeme

Das CAP-Theorem besagt, dass ein verteiltes System bei einer Netzpartition zwischen Konsistenz (alle Knoten liefern dieselben Daten) und Verfügbarkeit (jede Anfrage wird beantwortet) wählen muss. Ein einzelner MariaDB-Server ist nicht verteilt und daher nicht betroffen. Sobald Sie replizieren oder clustern, betrifft es Sie unmittelbar: asynchrone Replikation bevorzugt Verfügbarkeit (das Replikat kann veraltet sein), Galera Cluster bevorzugt Konsistenz (ein Knoten ohne Quorum nimmt keine Schreibvorgänge an).

2. Datenmodellierung STUFE 1

Das Datenmodell ist die teuerste Entscheidung im Lebenszyklus einer Anwendung, weil es am schwersten zu ändern ist. Eine schlechte Abfrage lässt sich in Minuten korrigieren. Ein schlechtes Modell verfolgt Sie über Jahre.

2.1 Vom Geschäftsprozess zum Entity-Relationship-Modell

Der Weg beginnt bei den Substantiven der Fachdomäne. In der Immobilienverwaltung: Objekt, Einheit, Eigentümer, Mieter, Mietvertrag, Zahlung, Abrechnung, Beschluss. Jedes Substantiv, das eigenständig existiert und identifizierbar ist, wird eine Entität und damit eine Tabelle. Die Verben dazwischen (ein Mieter *bewohnt* eine Einheit, ein Eigentümer *besitzt* Miteigentumsanteile) sind die Beziehungen.

BEZIEHUNGSTYP	BEISPIEL	UMSETZUNG
1:1	Einheit und Zählerplatz	Fremdschlüssel mit UNIQUE in einer der beiden Tabellen, oder Zusammenlegen
1:n	Objekt hat viele Einheiten	Fremdschlüssel <code>objekt_id</code> in <code>einheit</code>
n:m	Eigentümer besitzt mehrere Einheiten, Einheit hat mehrere Eigentümer	Zwischentabelle <code>eigentum(eigentuemer_id, einheit_id, anteil, ab, bis)</code>
Selbstbezug	Gesellschaft hält Beteiligung an Gesellschaft	Fremdschlüssel auf eigene Tabelle, oft mit Zwischentabelle für Anteile

2.2 Schlüsselwahl

Der Primärschlüssel hat in InnoDB eine Sonderrolle: Die Tabelle ist physisch nach ihm sortiert (Clustered Index, Kapitel 6). Deshalb gelten Regeln, die über reine Eindeutigkeit hinausgehen.

- **Kurz und monoton wachsend:** `BIGINT UNSIGNED AUTO_INCREMENT` ist der Standard. Neue Zeilen werden am Ende eingefügt, keine Seitenaufspaltungen, jeder Sekundärindex speichert den Primärschlüssel und bleibt klein.
- **Kein UUID v4 als Primärschlüssel** in großen Tabellen: zufällige Werte streuen Einfügungen über den gesamten Baum, verursachen Page Splits und blähen alle Sekundärindizes um 16 Byte pro Eintrag auf. Falls fachlich eine global eindeutige Kennung nötig ist: UUID v7 (zeitbasiert, sortierbar) oder UUID als separate UNIQUE-Spalte neben einem numerischen Primärschlüssel. MariaDB bietet seit 10.7 den Datentyp `UUID`, der intern kompakt speichert.
- **Natürliche Schlüssel** (Steuernummer, ISBN, Kennzeichen) sind verlockend, ändern sich aber häufiger als erwartet und sind in der Regel länger. Als UNIQUE-Constraint ja, als Primärschlüssel selten.
- **Zusammengesetzte Primärschlüssel** sind bei reinen Zwischentabellen (n:m) sinnvoll: `PRIMARY KEY (eigentuemer_id, einheit_id)`. Die Spaltenreihenfolge bestimmt, welche Zugriffe schnell sind.

2.3 Normalisierung

Normalisierung entfernt Redundanz, damit eine Tatsache genau an einer Stelle steht. Redundanz ist gefährlich, weil sie Widersprüche erzeugt: Wenn die Adresse eines Eigentümers in 14 Zeilen steht und Sie 13 davon ändern, hat Ihr System zwei Wahrheiten.

NORMALFORM	REGEL	VERSTOSS (BEISPIEL)	KORREKTUR
1NF	Jedes Feld enthält genau einen atomaren Wert, keine Wiederholgruppen	Spalte telefon = '0211..., 0172...'	Tabelle kontakt_telefon mit einer Zeile je Nummer
2NF	Jedes Nichtschlüsselattribut hängt vom ganzen Schlüssel ab	In eigentum(eigentuemer_id, einheit_id, eigentuemer_name) hängt der Name nur vom Eigentümer ab	Name in Tabelle eigentuemer
3NF	Kein Nichtschlüsselattribut hängt von einem anderen Nichtschlüsselattribut ab	einheit(plz, ort): Ort hängt von PLZ ab	Meist toleriert; streng: Tabelle plz_ort
BCNF	Verschärfte 3NF, jede Determinante ist Schlüsselkandidat	Selten praxisrelevant	Zerlegung

Ziel im OLTP-Betrieb ist 3NF. Höhere Normalformen (4NF, 5NF) sind akademisch relevant, praktisch selten. Der Kern jeder Normalisierungsentscheidung lautet: Was ist eine eigenständige Tatsache, und wo darf sie genau einmal stehen?

2.4 Bewusste Denormalisierung

Normalisierung optimiert Schreibkonsistenz. Lesezugriffe zahlen mit Joins. Wenn eine Abfrage 50-mal pro Sekunde über sieben Tabellen joint, kann gezielte Redundanz richtig sein. Sie muss aber dokumentiert und technisch abgesichert werden.

- **Berechnete Spalten:** bruttomiete DECIMAL(10,2) AS (kaltmiete + nebenkosten) STORED oder VIRTUAL. Der Server hält den Wert konsistent, man kann ihn indizieren.
- **Materialisierte Aggregate:** Tabelle objekt_kennzahlen mit Summen, gepflegt per Trigger oder nächtlichem Job. Vorteil: Reporting in Millisekunden. Nachteil: Verzögerung, zusätzlicher Code.
- **Historisierte Snapshots:** Die Anschrift auf einer Rechnung muss so bleiben, wie sie beim Rechnungsdatum war. Das ist keine Redundanz, sondern fachliche Korrektheit. Alternativ System-Versioned Tables (Kapitel 14).

2.5 Datentypen richtig wählen

Datentypen beeinflussen Speicherbedarf, Indexgröße, Vergleichsgeschwindigkeit und Korrektheit. Die Faustregel lautet: der kleinste Typ, der alle gültigen Werte sicher aufnimmt.

BEDARF	RICHTIG	FALSCH (HÄUFIG)	BEGRÜNDUNG
Geldbeträge	DECIMAL (12 , 2)	FLOAT, DOUBLE	Gleitkomma rundet: 0,1 + 0,2 ist nicht 0,3. In der Buchhaltung inakzeptabel.
Ganzzahl-ID	INT UNSIGNED (bis 4,29 Mrd.) oder BIGINT UNSIGNED	INT (11) mit Vorzeichen, VARCHAR	Die Zahl in Klammern ist nur Anzeigebreite. UNSIGNED verdoppelt den Wertebereich.

Zeitpunkt	DATETIME (0) oder TIMESTAMP	VARCHAR (19), Unix-Zeit in INT	Nur echte Datumstypen ermöglichen Datumsfunktionen und Bereichsindizes. TIMESTAMP endet 2038 und ist zeitzonenabhängig.
Kurzer Text	VARCHAR (n) mit realistischem n	VARCHAR (255) überall, TEXT	Temporäre Tabellen im Speicher reservieren die volle Breite. TEXT verhindert Speicher-Temptables.
Ja/Nein	TINYINT (1) bzw. BOOLEAN	VARCHAR (1) 'J' / 'N', ENUM	Ein Byte, eindeutige Semantik.
Status mit festen Werten	Nachschlagetabelle oder TINYINT mit Konstanten im Code	ENUM	ENUM ist schnell, aber jede Werteänderung ist ein ALTER TABLE.
Zeichensatz	utf8mb4, Kollation utf8mb4_unicode_ci oder uca1400_ai_ci	latin1, utf8 (3 Byte, kein Emoji)	utf8mb4 ist der einzige vollständige Unicode-Zeichensatz in MariaDB.

SPEICHER IST GESCHWINDIGKEIT

Jedes Byte, das Sie in einer Spalte sparen, sparen Sie in jeder Zeile, in jedem Index, der die Spalte enthält, im Buffer Pool, im Netzwerk und im Backup. Eine Tabelle mit 50 Millionen Zeilen und einer unnötig 8 statt 4 Byte breiten ID kostet 200 MB nur in der Tabelle, ohne Indizes. Kompakte Schemata passen in den Arbeitsspeicher, und Arbeitsspeicher ist etwa 100.000-mal schneller als eine Festplatte.

2.6 NULL bewusst einsetzen

NULL bedeutet "unbekannt" oder "nicht anwendbar", nicht "leer" und nicht Null. Vergleiche mit NULL ergeben weder wahr noch falsch, sondern NULL. `WHERE spalte = NULL` findet nie etwas, korrekt ist `IS NULL`. Aggregatfunktionen ignorieren NULL, `COUNT (spalte)` zählt anders als `COUNT (*)`. Deklarieren Sie Spalten `NOT NULL`, wo immer die Fachlichkeit es zulässt: Der Optimizer kann dann einfachere Pläne wählen, und Ihre Anwendung braucht weniger Sonderfälle.

2.7 Constraints als Schutzschicht

Alles, was der Server prüfen kann, sollte der Server prüfen. Anwendungscode hat Fehler, wird umgangen (Skripte, Importe, manuelle Korrekturen) und wechselt. Die Datenbank bleibt.

```
CREATE TABLE mietvertrag (  
  vertrag_id      BIGINT UNSIGNED NOT NULL AUTO_INCREMENT PRIMARY KEY,  
  einheit_id     INT UNSIGNED     NOT NULL,  
  mieter_id      INT UNSIGNED     NOT NULL,  
  beginn         DATE             NOT NULL,  
  ende           DATE             NULL,  
  kaltmiete      DECIMAL(10,2)    NOT NULL,  
  nebenkosten    DECIMAL(10,2)    NOT NULL DEFAULT 0,  
  bruttomiete    DECIMAL(10,2)    AS (kaltmiete + nebenkosten) STORED,  
  angelegt_am    DATETIME         NOT NULL DEFAULT CURRENT_TIMESTAMP,  
  CONSTRAINT fk_mv_einheit FOREIGN KEY (einheit_id) REFERENCES einheit(einheit_id),  
  CONSTRAINT fk_mv_mieter  FOREIGN KEY (mieter_id) REFERENCES mieter(mieter_id),  
  CONSTRAINT ck_mv_zeitraum CHECK (ende IS NULL OR ende > beginn),  
  CONSTRAINT ck_mv_miete   CHECK (kaltmiete >= 0 AND nebenkosten >= 0),  
  INDEX ix_mv_einheit_beginn (einheit_id, beginn)  
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;
```

MariaDB erzwingt CHECK-Constraints seit 10.2.1 tatsächlich (MySQL erst ab 8.0.16). Fremdschlüssel kosten bei jedem INSERT und DELETE eine Prüfung im Elternindex. Das ist in OLTP fast immer den Preis wert. In reinen Massenszenarien kann man sie temporär mit `SET foreign_key_checks = 0` abschalten, muss die Integrität dann aber selbst garantieren.

3. SQL von Grund auf

STUFE 1 BIS 2

SQL besteht aus wenigen Bausteinen, die sich nahezu beliebig kombinieren lassen. Wer die logische Auswertungsreihenfolge verinnerlicht, versteht auch, warum bestimmte Formulierungen schnell und andere langsam sind.

3.1 Logische Auswertungsreihenfolge

Sie schreiben `SELECT ... FROM ... WHERE ... GROUP BY ... HAVING ... ORDER BY ... LIMIT`. Der Server denkt in einer anderen Reihenfolge:

1. `FROM` und `JOIN`: Die Ausgangsmenge wird gebildet.
2. `WHERE`: Zeilen werden gefiltert, bevor gruppiert wird. Hier wirken Indizes am stärksten.
3. `GROUP BY`: Zeilen werden zu Gruppen zusammengefasst.
4. `HAVING`: Gruppen werden gefiltert. Bedingungen, die keine Aggregate brauchen, gehören in `WHERE`, nicht in `HAVING`.
5. `SELECT`: Ausdrücke werden berechnet, Aliasnamen entstehen (deshalb kann `WHERE` keinen `SELECT`-Alias verwenden).
6. `DISTINCT`, dann `ORDER BY`, dann `LIMIT`.

3.2 DDL: Strukturen anlegen und ändern

```
CREATE DATABASE hv CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
USE hv;

CREATE TABLE objekt (
  objekt_id INT UNSIGNED NOT NULL AUTO_INCREMENT PRIMARY KEY,
  bezeichnung VARCHAR(120) NOT NULL,
  strasse VARCHAR(120) NOT NULL,
  plz CHAR(5) NOT NULL,
  ort VARCHAR(80) NOT NULL,
  verwaltungsart ENUM('WEG','MIET','SEV') NOT NULL,
  UNIQUE KEY ux_objekt_adresse (strasse, plz)
);

ALTER TABLE objekt ADD COLUMN baujahr SMALLINT UNSIGNED NULL AFTER ort;
ALTER TABLE objekt MODIFY COLUMN bezeichnung VARCHAR(160) NOT NULL;
ALTER TABLE objekt ADD INDEX ix_objekt_plz (plz);
DROP INDEX ix_objekt_plz ON objekt;
RENAME TABLE objekt TO liegenschaft;
TRUNCATE TABLE tmp_import; -- leert Tabelle, setzt AUTO_INCREMENT zurück, nicht
transaktional
```

3.3 DML: Daten schreiben

```

INSERT INTO mieter (name, vorname, email) VALUES ('Beispiel', 'Anna', 'a@example.org');
INSERT INTO mieter (name, vorname) VALUES ('A','B'), ('C','D'), ('E','F');  --
Mehrzeilen-Insert, deutlich schneller

-- Einfügen oder aktualisieren (Upsert), benötigt PRIMARY oder UNIQUE KEY
INSERT INTO zaehlerstand (zaehler_id, datum, wert) VALUES (17, '2026-09-01', 4812)
ON DUPLICATE KEY UPDATE wert = VALUES(wert);

-- Einfügen, Konflikte ignorieren
INSERT IGNORE INTO tag (name) VALUES ('Balkon');

-- Update mit Join
UPDATE einheit e
JOIN objekt o ON o.objekt_id = e.objekt_id
SET e.verwalter_id = 3
WHERE o.plz LIKE '40%';

-- Löschen mit Limit (in Batches, um Sperren kurz zu halten)
DELETE FROM protokoll WHERE erstellt < NOW() - INTERVAL 2 YEAR LIMIT 5000;

-- Rückgabe der betroffenen Zeilen (MariaDB-Erweiterung)
DELETE FROM warteschlange WHERE status = 'fertig' RETURNING job_id;

```

3.4 Joins

JOIN	ERGEBNIS	TYPISCHE FRAGE
INNER JOIN	Nur Zeilen mit Treffer auf beiden Seiten	Welche Einheiten haben einen laufenden Mietvertrag?
LEFT JOIN	Alle Zeilen links, rechts NULL bei fehlendem Treffer	Alle Einheiten, mit Vertrag falls vorhanden (Leerstand erkennen)
LEFT JOIN ... WHERE r.id IS NULL	Anti-Join: nur Zeilen ohne Treffer	Einheiten ohne Vertrag
RIGHT JOIN	Spiegelbild von LEFT, in der Praxis meiden	Tabellen tauschen und LEFT verwenden
CROSS JOIN	Kartesisches Produkt	Kalender mal Einheiten für Soll-Matrix
Self Join	Tabelle mit sich selbst	Beteiligungsketten, Hierarchien

```

-- Leerstand zum Stichtag
SELECT e.einheit_id, o.bezeichnung, e.lage
FROM einheit e
JOIN objekt o ON o.objekt_id = e.objekt_id
LEFT JOIN mietvertrag mv
    ON mv.einheit_id = e.einheit_id
    AND mv.beginn <= '2026-09-01'
    AND (mv.ende IS NULL OR mv.ende >= '2026-09-01')
WHERE mv.vertrag_id IS NULL;

```

Beachten Sie den Unterschied: Bedingungen an der rechten Tabelle eines LEFT JOIN gehören in die ON-Klausel. In WHERE würden sie NULL-Zeilen entfernen und den LEFT JOIN faktisch zum INNER JOIN machen.

3.5 Aggregation

```
SELECT o.bezeichnung,
       COUNT(*)           AS einheiten,
       COUNT(mv.vertrag_id) AS vermietet,      -- NULL wird nicht gezählt
       SUM(mv.bruttomiete) AS soll_monat,
       ROUND(AVG(mv.kaltmiete / e.wohnflaeche), 2) AS miete_qm,
       GROUP_CONCAT(DISTINCT e.lage ORDER BY e.lage SEPARATOR ', ') AS lagen
FROM objekt o
JOIN einheit e ON e.objekt_id = o.objekt_id
LEFT JOIN mietvertrag mv ON mv.einheit_id = e.einheit_id AND mv.ende IS NULL
GROUP BY o.objekt_id, o.bezeichnung
HAVING vermietet < einheiten
ORDER BY soll_monat DESC;
```

Mit aktiviertem `ONLY_FULL_GROUP_BY` (Standard in neuen Installationen) müssen alle nicht aggregierten `SELECT`-Spalten in `GROUP BY` stehen oder funktional vom Gruppenschlüssel abhängen. Das ist keine Schikane, sondern verhindert zufällige Ergebnisse.

3.6 Unterabfragen und ihre Fallen

```
-- Skalare Unterabfrage
SELECT name, (SELECT COUNT(*) FROM mietvertrag mv WHERE mv.mieter_id = m.mieter_id) AS
vertraege
FROM mieter m;

-- IN mit Unterabfrage (Optimizer wandelt meist in Semi-Join um)
SELECT * FROM einheit WHERE objekt_id IN (SELECT objekt_id FROM objekt WHERE ort =
'Erkelenz');
```

```
-- EXISTS: oft klarer und gleich schnell
SELECT * FROM einheit e WHERE EXISTS (SELECT 1 FROM mietvertrag mv WHERE mv.einheit_id =
e.einheit_id);

-- NOT IN mit NULL-Falle: liefert leeres Ergebnis, wenn die Unterabfrage ein NULL
enthält
SELECT * FROM mieter WHERE mieter_id NOT IN (SELECT mieter_id FROM mahnung); -- riskant
SELECT * FROM mieter m WHERE NOT EXISTS (SELECT 1 FROM mahnung x WHERE x.mieter_id =
m.mieter_id); -- sicher
```

3.7 Common Table Expressions (CTE)

Eine CTE benennt eine Zwischenmenge und macht komplexe Abfragen lesbar. Rekursive CTEs lösen Hierarchien (Beteiligungsstrukturen, Stücklisten, Organigramme) in einer Abfrage.

```
-- Beteiligungskette von der Holding abwärts mit durchgerechneter Quote
WITH RECURSIVE kette AS (
  SELECT tochter_id, mutter_id, anteil, 1 AS ebene, anteil AS effektiv
  FROM beteiligung WHERE mutter_id = 1
  UNION ALL
  SELECT b.tochter_id, b.mutter_id, b.anteil, k.ebene + 1, k.effektiv * b.anteil
  FROM beteiligung b JOIN kette k ON b.mutter_id = k.tochter_id
  WHERE k.ebene < 10
)
SELECT g.name, k.ebene, ROUND(k.effektiv * 100, 2) AS quote_prozent
FROM kette k JOIN gesellschaft g ON g.id = k.tochter_id
ORDER BY k.ebene, g.name;
```

3.8 Window Functions

Fensterfunktionen berechnen Werte über eine Zeilenmenge, ohne die Zeilen zu verschmelzen. Sie ersetzen viele Self-Joins und korrelierte Unterabfragen und sind seit MariaDB 10.2 verfügbar.

```
SELECT z.mieter_id, z.datum, z.betrag,
       SUM(z.betrag) OVER (PARTITION BY z.mieter_id ORDER BY z.datum)      AS
saldo_kumuliert,
       LAG(z.datum) OVER (PARTITION BY z.mieter_id ORDER BY z.datum)      AS
vorherige_zahlung,
       ROW_NUMBER() OVER (PARTITION BY z.mieter_id ORDER BY z.datum DESC) AS rn,
       AVG(z.betrag) OVER (PARTITION BY z.mieter_id ORDER BY z.datum
                           ROWS BETWEEN 2 PRECEDING AND CURRENT ROW)      AS
gleitend_3
FROM zahlung z;

-- Neuester Datensatz je Gruppe (klassisches Problem, elegante Lösung)
SELECT * FROM (
  SELECT e.*, ROW_NUMBER() OVER (PARTITION BY einheit_id ORDER BY datum DESC) AS rn FROM
  zaehlerstand e
) t WHERE rn = 1;
```

FUNKTION	ZWECK
ROW_NUMBER, RANK, DENSE_RANK, NTILE	Numerierung und Rangfolge innerhalb der Partition
LAG, LEAD	Wert der vorherigen bzw. nächsten Zeile
FIRST_VALUE, LAST_VALUE, NTH_VALUE	Randwerte des Fensters
SUM, AVG, COUNT, MIN, MAX OVER	Laufende Summen, gleitende Durchschnitte
PERCENT_RANK, CUME_DIST, MEDIAN	Verteilungskennzahlen

3.9 Sichten, Prozeduren, Trigger, Events

Views kapseln Abfragen und Berechtigungen. Einfache Views werden vom Optimizer mit der äußeren Abfrage verschmolzen (ALGORITHM=MERGE), Views mit GROUP BY oder DISTINCT werden temporär materialisiert und können langsam sein. **Stored Procedures** bündeln Logik serverseitig, sparen Netzwerk-Roundtrips und sind für Batch-Verarbeitung sinnvoll; Geschäftslogik darin zu verstecken erschwert aber Versionierung und Tests. **Trigger** laufen bei INSERT, UPDATE oder DELETE automatisch, sind unsichtbar und verlängern jede Schreiboperation; für

Audit-Tabellen und Pflege abgeleiteter Werte akzeptabel, für Fachlogik nicht. **Events** sind der eingebaute Zeitplaner für wiederkehrende Aufgaben wie Archivierung; `event_scheduler=ON` muss gesetzt sein.

4. MariaDB-Architektur STUFE 2

Performance-Arbeit ohne Architekturverständnis ist Raten. Dieses Kapitel beschreibt, was zwischen dem Eintreffen einer Abfrage und dem Schreiben auf die Platte passiert.

4.1 Schichtenmodell

SCHICHT	AUFGABE	RELEVANTE STELLSCHRAUBEN
Verbindungsschicht	Nimmt TCP- oder Socket-Verbindungen an, authentifiziert, ein Thread je Verbindung (oder Thread Pool)	<code>max_connections</code> , <code>thread_cache_size</code> , <code>thread_handling=pool-of-threads</code>
Parser	Zerlegt SQL in einen Syntaxbaum, prüft Rechte	Prepared Statements sparen wiederholtes Parsen
Optimizer	Wählt Zugriffspfade, Join-Reihenfolge, Indexnutzung anhand von Statistiken und Kostenmodell	Indizes, <code>ANALYZE TABLE</code> , Histogramme, <code>optimizer_switch</code>
Executor	Führt den Plan aus, ruft die Storage Engine über eine Handler-Schnittstelle auf	Join Buffer, Sort Buffer, temporäre Tabellen
Storage Engine	Speichert und liest Zeilen, verwaltet Indizes, Sperren, Transaktionen, Caching	Engine-Wahl, InnoDB-Parameter
Dateisystem / OS	Blockgeräte, Page Cache, Dateisystem	I/O-Scheduler, Dateisystem, <code>innodb_flush_method</code>

Die Trennung zwischen Server und Storage Engine ist das Alleinstellungsmerkmal der MySQL-Familie. Sie können je Tabelle entscheiden, wie Daten gespeichert werden. In der Praxis ist InnoDB für 95 Prozent aller Tabellen die richtige Wahl.

4.2 Storage Engines im Überblick

ENGINE	TRANSAKTIONEN	SPERRGRANULARITÄT	EINSATZ
InnoDB	Ja, ACID, MVCC	Zeile	Standard für alle OLTP-Daten. Crash-sicher, Fremdschlüssel, Clustered Index.
Aria	Nein (crash-safe für Tabellenstruktur)	Tabelle	MariaDB-eigener MyISAM-Nachfolger, wird intern für temporäre Tabellen und Systemtabellen genutzt.
MyISAM	Nein	Tabelle	Historisch. Nicht crash-sicher. Für neue Projekte nicht verwenden.
MEMORY	Nein	Tabelle	Daten nur im RAM, gehen bei Neustart verloren. Für Nachschlagetabellen selten noch sinnvoll, InnoDB im Buffer Pool ist vergleichbar schnell.
ColumnStore	Eingeschränkt	Spaltenblöcke	Analytische Abfragen über sehr große Datenmengen, verteilt. Eigener Betriebsaufwand.

Spider	Ja (XA)	Remote	Föderiert Tabellen anderer Server, Sharding auf Engine-Ebene.
S3	Nein (read-only)	-	Archivtabellen in Objektspeicher auslagern.
CONNECT	Nein	-	Zugriff auf externe Dateien (CSV, JSON, ODBC) als Tabelle.
MyRocks	Ja	Zeile	LSM-Baum statt B-Baum, sehr hohe Kompression und Schreibdurchsatz, langsames Lesen. Für Log- und Ereignisdaten.

4.3 InnoDB im Detail

SEITEN UND TABLESPACES

InnoDB verwaltet Daten in Seiten (Pages) von 16 KB (`innodb_page_size`, nur bei Initialisierung änderbar). Jede Tabelle liegt standardmäßig in einer eigenen Datei (`innodb_file_per_table=ON`, Datei `tabelle.ibd`). Der System-Tablespace `ibdata1` enthält Datenwörterbuch-Reste, Undo-Informationen (sofern nicht in separaten Undo-Tablespaces) und den Change Buffer. Eine Seite ist die kleinste Einheit, die zwischen Platte und Speicher bewegt wird. Wer eine Zeile von 100 Byte liest, liest 16 KB.

BUFFER POOL

Der Buffer Pool ist der zentrale Cache für Daten- und Indexseiten und der mit Abstand wichtigste Performancefaktor. Jede Lese- und Schreiboperation geht über ihn. Ist eine Seite nicht vorhanden (Miss), wird sie von der Platte geladen. Änderungen werden zunächst nur im Buffer Pool vorgenommen (die Seite wird "dirty"), das Redo Log sichert die Änderung ab, und ein Hintergrundprozess schreibt die Seite später zurück (Flush). Der Pool ist als LRU-Liste mit einem Young- und einem Old-Bereich organisiert, sodass ein einmaliger Tabellenscan die häufig genutzten Seiten nicht verdrängt.

```
SHOW ENGINE INNODB STATUS\G          -- Abschnitt BUFFER POOL AND MEMORY
SHOW GLOBAL STATUS LIKE 'Innodb_buffer_pool_read%';
-- Trefferquote = 1 - (Innodb_buffer_pool_reads / Innodb_buffer_pool_read_requests)
-- Ziel im OLTP: > 99 %. Unter 95 % ist der Pool zu klein oder das Working Set zu groß.
```

REDO LOG (WRITE-AHEAD LOG)

Jede Änderung wird als kompakter Logeintrag in das Redo Log geschrieben, bevor die Transaktion als bestätigt gilt. Sequentielles Schreiben ist um Größenordnungen schneller als das verstreute Zurückschreiben der Datenseiten. Nach einem Absturz spielt InnoDB das Redo Log nach und stellt alle bestätigten Änderungen wieder her. Seit MariaDB 10.5 gibt es eine einzige Datei `ib_logfile0`; die Größe (`innodb_log_file_size`) ist ab 10.9 zur Laufzeit änderbar. Zu kleine Redo Logs führen zu aggressivem Flushing und Schreibstaus (Checkpoint-Druck), zu große verlängern die Recovery-Zeit.

UNDO LOG UND MVCC

Vor jeder Änderung sichert InnoDB die alte Version der Zeile im Undo Log. Das dient zwei Zwecken: Rollback einer Transaktion und Multi-Version Concurrency Control. Eine lesende Transaktion sieht den Zustand zum Zeitpunkt ihres Beginns (Snapshot); wenn eine Zeile inzwischen geändert wurde, rekonstruiert InnoDB die alte Version aus dem Undo Log. Deshalb blockieren Leser keine Schreiber und umgekehrt. Der Preis: Langlaufende Lesetransaktionen halten Undo-Einträge am Leben, die History List wächst, und der Purge-Thread kommt nicht hinterher. Ein Wert von `History list length` im sechsstelligen Bereich in `SHOW ENGINE INNODB STATUS` ist ein Warnsignal.

DOUBLEWRITE BUFFER

Weil eine 16-KB-Seite in mehreren 4-KB-Blöcken geschrieben wird, könnte ein Absturz eine halb geschriebene Seite hinterlassen (Torn Page). Der Doublewrite Buffer schreibt jede Seite erst in einen Puffer und dann an den Zielort. Auf Dateisystemen mit atomaren Schreibvorgängen (ZFS, oder bestimmte NVMe-Konfigurationen) kann er abgeschaltet werden; im Zweifel eingeschaltet lassen.

CHANGE BUFFER UND ADAPTIVE HASH INDEX

Der Change Buffer puffert Änderungen an Sekundärindexseiten, die nicht im Buffer Pool sind, und wendet sie später gebündelt an. Der Adaptive Hash Index baut für sehr häufig gleichartig zugegriffene Seiten automatisch einen Hash-Zugriff. Beide helfen bei bestimmten Lasten und schaden bei anderen; auf schnellen SSDs und bei hohem Parallelismus ist das Abschalten des Adaptive Hash Index (`innodb_adaptive_hash_index=OFF`) oft ein Gewinn. Messen statt glauben.

4.4 Der Weg eines UPDATE

1. Verbindung nimmt Statement an, Parser erzeugt Syntaxbaum, Rechteprüfung.
2. Optimizer wählt Index zum Auffinden der Zeile.
3. InnoDB sucht die Seite im Buffer Pool, lädt sie bei Bedarf von der Platte.
4. Exklusive Zeilensperre wird gesetzt (und ggf. Gap Lock, Kapitel 5).
5. Alte Zeilenversion wird ins Undo Log geschrieben.
6. Zeile wird in der Seite geändert, Seite wird als dirty markiert.
7. Redo-Eintrag wird in den Log Buffer geschrieben.
8. Bei COMMIT: Log Buffer wird in die Redo-Datei geschrieben und je nach `innodb_flush_log_at_trx_commit` per fsync bestätigt. Binlog-Eintrag wird geschrieben (Group Commit koordiniert beides).
9. Sperren werden freigegeben, Client erhält OK.
10. Später: Page Cleaner schreibt die dirty Page an ihren Ort, Purge räumt Undo-Einträge auf.

Jeder Schritt hat eigene Engpässe. Schritt 3 ist Buffer-Pool-Größe, Schritt 4 ist Sperrkonflikt, Schritt 8 ist fsync-Latenz des Speichermediums. Diagnose heißt herauszufinden, welcher Schritt wartet.

4.5 Binärlog

Das Binary Log ist serverseitig (nicht Engine-seitig) und protokolliert alle Datenänderungen. Es ist die Grundlage für Replikation und Point-in-Time-Recovery. Format ROW (Standard seit 10.2) protokolliert die tatsächlich geänderten Zeilen und ist deterministisch; STATEMENT protokolliert das SQL und ist bei nichtdeterministischen Funktionen unsicher; MIXED wechselt automatisch. Wer weder repliziert noch PITR braucht, kann das Binlog abschalten und spart Schreiblast; im professionellen Betrieb ist es Pflicht.

4.6 Speicherverbrauch verstehen

Der Speicherbedarf des Servers setzt sich aus globalen Puffern (einmal) und Verbindungspuffern (je Verbindung) zusammen. Eine grobe Formel:

```
Gesamt ≈ innodb_buffer_pool_size + innodb_log_buffer_size + key_buffer_size  
(MyISAM/Aria)  
      + max_connections × (sort_buffer_size + join_buffer_size + read_buffer_size  
                           + read_rnd_buffer_size + tmp_table_size + thread_stack)
```

Die Verbindungspuffer werden nur bei Bedarf und meist nicht in voller Höhe belegt, dennoch ist die Formel der Grund, warum `sort_buffer_size = 256M` bei 500 Verbindungen den Server in den Swap treibt. Verbindungspuffer klein halten, Buffer Pool groß.

5. Transaktionen, Isolation und Sperren

STUFE 2 BIS 3

5.1 Transaktionssteuerung

```
START TRANSACTION;      -- oder BEGIN
UPDATE konto SET saldo = saldo - 500 WHERE konto_id = 1;
UPDATE konto SET saldo = saldo + 500 WHERE konto_id = 2;
COMMIT;                 -- oder ROLLBACK

SET autocommit = 0;     -- jede Sitzung: implizite Transaktion bis COMMIT
SAVEPOINT vor_buchung; ROLLBACK TO SAVEPOINT vor_buchung;
```

Standardmäßig ist autocommit=1: Jedes einzelne Statement ist eine Transaktion. Das ist korrekt, aber bei Massenimporten teuer, weil jeder INSERT einen eigenen fsync auslöst. 100.000 einzelne INSERTs mit Autocommit dauern Minuten; dieselben Zeilen in einer Transaktion oder als Mehrzeilen-INSERT dauern Sekunden. DDL-Statements (CREATE, ALTER, DROP) lösen ein implizites COMMIT aus.

5.2 Die Anomalien, die Isolation verhindert

ANOMALIE	BESCHREIBUNG
Dirty Read	Transaktion A liest Daten, die B geändert, aber noch nicht bestätigt hat. B rollt zurück. A hat Daten gesehen, die nie existierten.
Non-Repeatable Read	A liest eine Zeile zweimal, dazwischen ändert B sie und bestätigt. A sieht zwei verschiedene Werte.
Phantom Read	A führt eine Bereichsabfrage zweimal aus, dazwischen fügt B eine passende Zeile ein. A sieht beim zweiten Mal eine Zeile mehr.
Lost Update	A und B lesen denselben Wert, beide addieren etwas und schreiben zurück. Eine Änderung geht verloren.
Write Skew	Zwei Transaktionen prüfen dieselbe Bedingung und schreiben beide, obwohl nur eine dürfte (z. B. letzter Termin doppelt vergeben).

5.3 Isolationsstufen in InnoDB

STUFE	DIRTY READ	NON-REPEATABLE	PHANTOM	VERHALTEN IN INNODB
READ UNCOMMITTED	möglich	möglich	möglich	Nie im Produktivbetrieb verwenden.
READ COMMITTED	nein	möglich	möglich	Snapshot je Statement. Keine Gap Locks (außer bei Fremdschlüssel- und Duplikatprüfung). Weniger Deadlocks, höherer Parallelismus. Erfordert Binlog-Format ROW.
REPEATABLE READ (Standard)	nein	nein	weitgehend nein	Snapshot je Transaktion. InnoDB verhindert Phantome durch Gap Locks bei sperrenden Lesevorgängen und Schreibvorgängen. Reine SELECTs sperren nichts.

SERIALIZABLE	nein	nein	nein	Jedes SELECT wird zu <code>SELECT ... LOCK IN SHARE MODE</code> . Sehr sicher, sehr geringer Parallelismus.
--------------	------	------	------	---

```
SET SESSION TRANSACTION ISOLATION LEVEL READ COMMITTED;  -- für die Sitzung
SET GLOBAL transaction_isolation = 'READ-COMMITTED';      -- Serverweit (my.cnf:
transaction_isolation)
```

PRAXISEMPFEHLUNG

Viele erfahrene Betreiber setzen Webanwendungen auf READ COMMITTED, weil Gap Locks unter REPEATABLE READ eine der häufigsten Deadlock-Ursachen sind und weil das Statement-weise Snapshot-Verhalten dem entspricht, was Entwickler intuitiv erwarten (und was PostgreSQL und Oracle standardmäßig tun). Für Abrechnungsläufe, die innerhalb einer Transaktion konsistente Summen über viele Tabellen bilden, ist REPEATABLE READ die richtige Wahl. Die Stufe kann je Sitzung gesetzt werden; nutzen Sie das.

5.4 Sperrtypen

SPERRE	WIRD GESETZT BEI	WIRKUNG
Shared (S) Zeilen Sperre	<code>SELECT ... LOCK IN SHARE MODE</code> , Fremdschlüsselprüfung	Andere dürfen lesen, nicht ändern
Exclusive (X) Zeilen Sperre	<code>UPDATE</code> , <code>DELETE</code> , <code>SELECT ... FOR UPDATE</code>	Niemand sonst darf die Zeile sperren
Gap Lock	Bereichszugriff unter REPEATABLE READ	Sperrt die Lücke zwischen Indexwerten, verhindert Einfügungen
Next-Key Lock	Kombination aus Zeilen- und Gap Lock	Standardmechanismus bei Index-Bereichsscans
Insert Intention Lock	<code>INSERT</code>	Signalisiert Einfügeabsicht in eine Lücke
Intention Locks (IS, IX)	Jeder Zeilenzugriff	Tabellenebene, koordiniert mit Tabellensperren
Metadata Lock (MDL)	Jede Nutzung einer Tabelle	Verhindert DDL, während Transaktionen die Tabelle nutzen. Ursache für hängende <code>ALTER TABLE</code> .
AUTO-INC Lock	<code>INSERT</code> in AUTO_INCREMENT-Tabellen	<code>innodb_autoinc_lock_mode=2</code> (interleaved) ist am schnellsten, erfordert ROW-Binlog

Wichtig: InnoDB sperrt Indexeinträge, nicht Zeilen im abstrakten Sinn. Fehlt ein passender Index für die WHERE-Bedingung eines UPDATE, muss InnoDB alle Zeilen prüfen und sperrt dabei jede einzelne. Ein UPDATE ohne Index kann so eine ganze Tabelle blockieren. Das ist einer der wichtigsten Gründe, warum Indizes auch für Schreiboperationen zählen.

5.5 Deadlocks

Ein Deadlock entsteht, wenn A auf eine Sperre von B wartet und B auf eine Sperre von A. InnoDB erkennt das sofort und bricht die Transaktion ab, die weniger Undo-Arbeit verursacht (Fehler 1213). Deadlocks sind kein Bug, sondern normal; die Anwendung muss die Transaktion wiederholen können.

```
SHOW ENGINE INNODB STATUS\G          -- Abschnitt LATEST DETECTED DEADLOCK
SET GLOBAL innodb_print_all_deadlocks = ON;  -- alle Deadlocks ins Error Log
SELECT * FROM information_schema.INNODB_LOCKS;      -- bis 10.5
SELECT * FROM performance_schema.data_locks;      -- ab 10.5, wenn Performance
Schema aktiv
SELECT * FROM information_schema.INNODB_TRX WHERE trx_state = 'LOCK WAIT';
```

Strategien zur Vermeidung: Zeilen immer in derselben Reihenfolge sperren (z. B. nach Primärschlüssel sortiert), Transaktionen kurz halten, keine Benutzerinteraktion innerhalb einer offenen Transaktion, passende Indizes für alle UPDATE/DELETE-Bedingungen, READ COMMITTED erwägen, SELECT ... FOR UPDATE gezielt einsetzen, wenn ein späteres UPDATE ohnehin folgt (verhindert die Eskalation von S auf X).

5.6 Lock Waits und Timeouts

`innodb_lock_wait_timeout` (Standard 50 Sekunden) bestimmt, wie lange eine Transaktion auf eine Sperre wartet, bevor sie mit Fehler 1205 abbricht. In interaktiven Anwendungen sind 50 Sekunden viel zu lang; 5 bis 10 Sekunden sind üblich. `lock_wait_timeout` (Standard 86400 Sekunden) gilt für Metadata Locks und ist der Grund, warum ein ALTER TABLE hinter einer vergessenen offenen Transaktion einen Tag lang hängen und dabei alle nachfolgenden Abfragen auf die Tabelle blockieren kann. Vor jedem DDL auf Produktivsystemen: SELECT * FROM information_schema.INNODB_TRX prüfen und `lock_wait_timeout` in der Sitzung auf wenige Sekunden setzen.

5.7 Optimistisches Sperren in der Anwendung

Zwischen Anzeigen eines Datensatzes und dem Speichern durch den Nutzer liegen Minuten. Keine Datenbanksperre darf so lange leben. Die Lösung ist eine Versionsspalte: UPDATE ... SET ..., version = version + 1 WHERE id = ? AND version = ?. Ist die Anzahl betroffener Zeilen 0, hat jemand zwischenzeitlich gespeichert, und die Anwendung meldet den Konflikt. Das ist der Standardmechanismus in allen gängigen ORMs.

6. Indexierung

STUFE 2 BIS 3

Indizes sind der Hebel mit dem größten Effekt: Eine Abfrage, die eine Sekunde dauert, braucht mit dem richtigen Index eine Millisekunde. Sie sind aber kein kostenloses Gut. Jeder Index muss bei jedem INSERT, UPDATE der indizierten Spalte und DELETE mitgepflegt werden und belegt Speicher im Buffer Pool.

6.1 Der B+-Baum

Alle regulären InnoDB-Indizes sind B+-Bäume. Ein B+-Baum ist ein balancierter Baum, dessen innere Knoten nur Schlüssel und Verweise enthalten und dessen Blätter die Daten (oder Verweise darauf) sortiert und miteinander verkettet halten. Bei 16-KB-Seiten und einem Schlüssel von wenigen Byte passen mehrere hundert bis über tausend Einträge in eine Seite. Ein Baum mit drei Ebenen adressiert dadurch bereits hunderte Millionen Zeilen; jede Punktsuche kostet drei Seitenzugriffe, von denen die oberen zwei praktisch immer im Buffer Pool liegen.

Daraus folgt: Ein B+-Baum ist hervorragend für Gleichheit (=), Bereiche (<, >, BETWEEN), Präfixe (LIKE 'Mül%'), Sortierung (ORDER BY in Indexreihenfolge) und Min/Max. Er hilft nicht bei Suffixen (LIKE '%ller'), bei Funktionen auf der Spalte (YEAR(datum) = 2026) und bei Ungleichheit (!=), weil in allen drei Fällen die Sortierung des Baums nicht ausgenutzt werden kann.

6.2 Clustered Index: Die Tabelle ist der Index

In InnoDB gibt es keine getrennte Datenablage. Die Tabelle selbst ist ein B+-Baum, sortiert nach dem Primärschlüssel, dessen Blätter die vollständigen Zeilen enthalten. Das nennt man Clustered Index. Fehlt ein Primärschlüssel, nimmt InnoDB den ersten UNIQUE NOT NULL Index; fehlt auch der, erzeugt es einen versteckten 6-Byte-Schlüssel. Beides ist schlecht: Legen Sie immer einen expliziten Primärschlüssel an.

Konsequenzen des Clustered Index:

- Zugriff über Primärschlüssel ist der schnellste mögliche Zugriff (ein Baumabstieg, Zeile liegt im Blatt).
- Bereichsabfragen über den Primärschlüssel lesen physisch benachbarte Zeilen.
- Jeder Sekundärindex speichert in seinen Blättern den Primärschlüssel als Verweis. Ein Zugriff über einen Sekundärindex kostet daher zwei Baumabstiege: erst im Sekundärindex den Primärschlüssel finden, dann im Clustered Index die Zeile. Ein breiter Primärschlüssel macht alle Sekundärindizes breit.
- Zufällige Primärschlüssel (UUID v4) erzeugen zufällige Einfügepositionen, Page Splits und fragmentierte Bäume.

6.3 Sekundärindizes und der Covering Index

Wenn ein Sekundärindex alle Spalten enthält, die eine Abfrage benötigt (in WHERE, SELECT, ORDER BY), muss InnoDB den zweiten Abstieg in den Clustered Index nicht durchführen. Das ist ein Covering Index; EXPLAIN zeigt Using index. Für hochfrequente Abfragen ist das der stärkste einzelne Optimierungsschritt nach dem Anlegen eines Index überhaupt.

```
-- Abfrage: SELECT mieter_id, bruttomiete FROM mietvertrag WHERE einheit_id = ? AND ende IS NULL
-- Index:   (einheit_id, ende, mieter_id, bruttomiete) -> vollständig covering
ALTER TABLE mietvertrag ADD INDEX ix_mv_cover (einheit_id, ende, mieter_id, bruttomiete);
```

6.4 Zusammengesetzte Indizes und die Präfixregel

Ein Index über (a, b, c) ist sortiert nach a, innerhalb gleicher a nach b, innerhalb gleicher a und b nach c. Er kann genutzt werden für Bedingungen auf a, auf a, b und auf a, b, c, nicht aber für Bedingungen nur auf b oder nur auf c. Das ist die Leftmost-Prefix-Regel. Ein Index auf (a, b, c) macht separate Indizes auf (a) und (a, b) überflüssig.

Reihenfolgeregel für die Spalten:

1. Spalten mit Gleichheitsbedingung zuerst, beginnend mit der selektivsten (viele verschiedene Werte).
2. Danach höchstens eine Spalte mit Bereichsbedingung; ab der ersten Bereichsbedingung kann der Index für weitere Spalten nicht mehr zum Filtern genutzt werden.
3. Alternativ nach der Gleichheitsspalte die Spalte aus ORDER BY, um den Sortierschritt zu sparen.
4. Zum Schluss Spalten, die nur für Covering benötigt werden.

```
-- WHERE objekt_id = ? AND status = 'offen' AND faellig < ? ORDER BY faellig
-- Guter Index: (objekt_id, status, faellig)  Gleichheit, Gleichheit, Bereich =
Sortierung
-- Schlechter Index: (faellig, objekt_id, status)  Bereich zuerst blockiert die Nutzung
der anderen Spalten
```

6.5 Kardinalität und Selektivität

Kardinalität ist die Anzahl verschiedener Werte in einer Spalte. Selektivität ist Kardinalität geteilt durch Zeilenzahl. Ein Index auf einer Spalte mit zwei Werten (Geschlecht, Aktiv-Flag) hat eine Selektivität nahe null und wird vom Optimizer meist zu Recht ignoriert, weil ein Tabellenscan billiger ist als eine Million Sekundärindex-Nachschläge. Ausnahme: stark schiefe Verteilung. Wenn nur 0,1 Prozent der Zeilen den Status "offen" haben und genau diese gesucht werden, ist der Index Gold wert. Der Optimizer weiß das nur, wenn Histogramme vorhanden sind (Kapitel 7).

```
SELECT COUNT(DISTINCT status) / COUNT(*) AS selektivitaet FROM forderung;
SHOW INDEX FROM forderung;  -- Spalte Cardinality ist eine Schätzung, ANALYZE TABLE
aktualisiert sie
```

6.6 Weitere Indexarten

TYP	ZWECK	HINWEISE
UNIQUE	Eindeutigkeit erzwingen, gleichzeitig Suchindex	NULL-Werte gelten als verschieden, mehrere NULL erlaubt
Präfixindex INDEX (spalte(20))	Lange Textspalten teilindizieren	Kann nicht covering sein, nicht für ORDER BY
FULLTEXT	Volltextsuche in Textspalten, MATCH ... AGAINST	Invertierter Index, boolescher und natürlichsprachlicher Modus, innodb_ft_min_token_size
SPATIAL	Geometrische Daten (R-Baum)	Für Umkreissuchen auf Koordinaten
Index auf virtueller Spalte	Funktionsergebnisse indizieren	MariaDB-Ersatz für funktionale Indizes, siehe unten

Descending Index	INDEX (datum DESC)	Seit 10.8 echte absteigende Indizes, relevant für gemischte ORDER BY
Hash (MEMORY Engine)	Nur Gleichheit	Nicht in InnoDB, außer Adaptive Hash Index intern
VECTOR	Ähnlichkeitssuche für Embeddings	MariaDB 11.7+, siehe Kapitel 14

```
-- Funktion in WHERE verhindert Indexnutzung:
SELECT * FROM zahlung WHERE YEAR(datum) = 2026; -- Tabellenscan
SELECT * FROM zahlung WHERE datum >= '2026-01-01' AND datum < '2027-01-01'; --
Bereichsscan, Index nutzbar

-- Funktionaler Index über virtuelle Spalte
ALTER TABLE mieter ADD COLUMN email_lower VARCHAR(255) AS (LOWER(email)) VIRTUAL,
ADD INDEX ix_mieter_email_lower (email_lower);
SELECT * FROM mieter WHERE email_lower = LOWER(?);
```

6.7 Anti-Muster

- **Index auf jede Spalte:** Schreibkosten steigen linear mit der Indexzahl, der Optimizer verliert Zeit bei der Planwahl, der Buffer Pool füllt sich mit Indexseiten, die niemand nutzt.
- **Redundante Indizes:** (a) neben (a, b). Der erste ist überflüssig. `sys.schema_redundant_indexes` listet sie auf.
- **Ungenutzte Indizes:** `sys.schema_unused_indexes` (benötigt aktives Performance Schema, Serverlaufzeit von Wochen als Basis). Löschen, nicht ansammeln.
- **Implizite Typumwandlung:** `WHERE kundennummer = 4711` auf einer VARCHAR-Spalte wandelt jede Zeile in eine Zahl um und ignoriert den Index. Typen im Vergleich müssen übereinstimmen, auch bei Joins (INT auf BIGINT ist unkritisch, VARCHAR auf INT ist fatal, unterschiedliche Kollationen ebenfalls).
- **OR über verschiedene Spalten:** `WHERE a = 1 OR b = 2` kann der Optimizer nur mit Index Merge lösen, was oft langsam ist. Alternative: UNION zweier Abfragen mit je einem Index.
- **Führendes Wildcard:** `LIKE '%str%'` ist immer ein Scan. Für Textsuche: FULLTEXT oder eine externe Suchmaschine.
- **Negation:** `NOT IN`, `!=`, `NOT LIKE` nutzen Indizes kaum. Positiv formulieren, wo möglich.

6.8 Indexpflege

InnoDB-Indizes fragmentieren durch Löschungen und Änderungen. `OPTIMIZE TABLE` (in InnoDB ein `ALTER TABLE ... FORCE`, der die Tabelle neu aufbaut) beseitigt das, sperrt aber während des Aufbaus (Online-fähig mit `ALGORITHM=INPLACE`, dennoch I/O-intensiv). In den meisten OLTP-Systemen ist das nur nach großen Löschaktionen nötig. `ANALYZE TABLE` aktualisiert die Statistiken und ist billig; InnoDB tut das mit `innodb_stats_auto_recalc` auch selbst, wenn sich mehr als 10 Prozent der Zeilen ändern.

7. Query-Optimierung STUFE 3

Der Optimizer wählt anhand eines Kostenmodells und Statistiken den Ausführungsplan mit den geringsten geschätzten Kosten. Er ist gut, aber nicht allwissend. Ihre Aufgabe ist, seine Entscheidung zu lesen, zu verstehen und ihm bei Bedarf bessere Grundlagen zu geben.

7.1 EXPLAIN lesen

```
EXPLAIN SELECT m.name, mv.bruttomiete
FROM mietvertrag mv JOIN mieter m ON m.mieter_id = mv.mieter_id
WHERE mv.einheit_id = 815 AND mv.ende IS NULL;
```

SPALTE	BEDEUTUNG	WORAUF ACHTEN
id / select_type	Nummer und Art des SELECT (SIMPLE, PRIMARY, SUBQUERY, DERIVED, MATERIALIZED)	DEPENDENT SUBQUERY wird je äußerer Zeile ausgeführt: teuer
table	Tabelle in Ausführungsreihenfolge (erste Zeile wird zuerst gelesen)	Reihenfolge ist die Join-Reihenfolge des Optimizers
type	Zugriffsart	Rangfolge von gut nach schlecht: system, const, eq_ref, ref, range, index, ALL
possible_keys / key	Kandidaten und gewählter Index	NULL in key bei großer Tabelle: fehlender Index
key_len	Genutzte Byte des Index	Zeigt, wie viele Spalten eines zusammengesetzten Index tatsächlich genutzt werden
ref	Womit der Index verglichen wird	const oder Spalte der vorherigen Tabelle
rows	Geschätzte zu prüfende Zeilen	Multipliziert über alle Zeilen ergibt die Gesamtarbeit
filtered	Geschätzter Anteil in Prozent, der nach WHERE übrig bleibt	Kleine Werte bei type ALL bedeuten: viel gelesen, wenig gebraucht
Extra	Zusatzinformationen	Siehe folgende Tabelle

EXTRA-WERT	BEDEUTUNG	BEWERTUNG
Using index	Covering Index, kein Zugriff auf Clustered Index	Sehr gut
Using index condition	Index Condition Pushdown: Filter wird in der Engine ausgewertet	Gut
Using where	Filter nach dem Zugriff auf Serverebene	Normal; kritisch nur zusammen mit type ALL
Using temporary	Temporäre Tabelle für GROUP BY oder DISTINCT	Prüfen, ob Index die Gruppierung liefern kann
Using filesort	Sortierung nicht über Index (trotz Name im Speicher, wenn klein genug)	Bei großen Mengen Index für ORDER BY anlegen

Using join buffer (Block Nested Loop / hash join)	Join ohne Index auf der inneren Tabelle	Fehlender Index auf der Join-Spalte
Range checked for each record	Kein brauchbarer Index, Prüfung je Zeile	Schlecht
Using index for group-by	Loose Index Scan	Sehr gut
Rowid-ordered scan / Key-ordered scan	Multi-Range Read: Zugriffe werden nach physischer Position sortiert	Gut bei Bereichsscans auf Sekundärindizes

7.2 ANALYZE: Schätzung gegen Wirklichkeit

`ANALYZE SELECT ...` (MariaDB-spezifisch, ab 10.1) führt die Abfrage tatsächlich aus und zeigt neben den Schätzungen die realen Werte `r_rows` und `r_filtered`. Weichen `rows` und `r_rows` um Größenordnungen ab, sind die Statistiken veraltet oder die Verteilung ist schief. `ANALYZE FORMAT=JSON` liefert zusätzlich Laufzeiten je Schritt (`r_total_time_ms`) und ist das wichtigste Diagnosewerkzeug für komplexe Abfragen.

```
ANALYZE FORMAT=JSON SELECT ... \G
-- Achten auf: "r_total_time_ms" je Tabelle, "r_rows" gegen "rows", "r_filtered" niedrig
bei "table_scan"
SHOW WARNINGS; -- nach EXPLAIN EXTENDED: zeigt die vom Optimizer umgeschriebene Abfrage
```

7.3 Wie der Optimizer arbeitet

1. **Umschreibung:** Unterabfragen werden zu Semi-Joins, Views werden verschmolzen, Konstanten propagiert, Bedingungen vereinfacht.
2. **Zugriffspfade je Tabelle:** Für jede Tabelle werden Kosten von Tabellenscan, Index-Range-Scan und Ref-Zugriff geschätzt (Statistiken, Histogramme, Kostenkonstanten).
3. **Join-Reihenfolge:** Für n Tabellen gibt es $n!$ Reihenfolgen. Der Optimizer sucht greedy mit Tiefe `optimizer_search_depth` (Standard 62 = automatisch) und bricht Zweige mit `optimizer_prune_level` ab. Bei mehr als etwa 10 Tabellen kann die Planung selbst messbar Zeit kosten.
4. **Join-Methode:** Nested Loop mit Index (Standard), Block Nested Loop mit Join Buffer, Hash Join (in MariaDB seit 5.3 als Block Hash Join über `join_cache_level`), Batched Key Access für MRR.
5. **Nachbearbeitung:** Sortierung, Gruppierung, LIMIT-Optimierung.

7.4 Statistiken und Histogramme

InnoDB liefert Kardinalitätsschätzungen aus Stichproben (`innodb_stats_persistent_sample_pages`, Standard 20 Seiten). Für schief verteilte Daten reicht das nicht: Der Optimizer nimmt Gleichverteilung an. MariaDB hat seit 10.0 Engine-unabhängige Statistiken mit Histogrammen in `mysql.column_stats`, die seit 10.4 standardmäßig genutzt werden (`use_stat_tables=PREFERABLY_FOR_QUERIES`) und seit 10.8 als JSON-Histogramme mit 254 Buckets vorliegen.

```
ANALYZE TABLE forderung PERSISTENT FOR COLUMNS (status, faellig) INDEXES ();
SELECT * FROM mysql.column_stats WHERE table_name = 'forderung';
SET GLOBAL histogram_size = 254; -- Standard seit 10.4 bereits 254
```

7.5 Häufige Fehler und ihre Korrektur

FEHLER	PROBLEM	KORREKTUR
SELECT *	Verhindert Covering, überträgt unnötige Daten, bricht bei Schemaänderung	Spalten explizit nennen
OFFSET 100000 für Paginierung	Der Server liest und verwirft 100.000 Zeilen	Keyset-Paginierung: WHERE id > :letzte_id ORDER BY id LIMIT 50
COUNT (*) über große Tabelle als Seitenzähler	InnoDB muss zählen (kein Zeilenzähler wegen MVCC)	Schätzung aus information_schema.TABLES, Zähltable, oder gar nicht anzeigen
ORDER BY RAND()	Sortiert die gesamte Tabelle	Zufällige ID im Anwendungscode ziehen
N+1-Abfragen (ORM)	Eine Abfrage je Zeile der Vorabfrage, 1.000 Roundtrips	Eager Loading, JOIN oder WHERE id IN (...)
Korrelierte Unterabfrage in SELECT	Wird je Zeile ausgeführt	JOIN mit vorab aggregierter Derived Table oder Window Function
Funktion auf indizierter Spalte	Index unbrauchbar	Bedingung umformen oder virtuelle Spalte indizieren
Riesige IN-Listen (10.000 Werte)	Parsezeit, Planungszeit, Sperrbreite	Werte in temporäre Tabelle laden und joinen
DISTINCT zur Reparatur falscher Joins	Verdeckt Kardinalitätsfehler, kostet Sortierung	Join-Bedingung korrigieren, EXISTS verwenden
Implizite Konvertierung im JOIN	Unterschiedliche Typen oder Kollationen der Join-Spalten	Typen und Kollationen der Fremdschlüssel exakt angleichen
Update in Schleife statt Set-Operation	Roundtrips, Sperren, Logvolumen	Ein UPDATE mit JOIN oder CASE

7.6 Optimizer steuern

```
-- Index erzwingen (Ausnahme, nicht Regel; dokumentieren, warum)
SELECT ... FROM forderung FORCE INDEX (ix_forderung_status_faellig) WHERE ...;
SELECT ... FROM forderung IGNORE INDEX (ix_schlecht) WHERE ...;
SELECT STRAIGHT_JOIN ... FROM a JOIN b ...; -- Join-Reihenfolge wie geschrieben

-- Optimizer-Features je Sitzung schalten
SET SESSION optimizer_switch = 'rowid_filter=off';
SET SESSION join_cache_level = 8; -- alle Join-Cache-Varianten inkl. Hash
Join erlauben
SET SESSION optimizer_use_condition_selectivity = 4; -- Histogramme voll nutzen
(Standard ab 10.4)
```

Hints sind ein Eingeständnis, dass der Optimizer eine Information nicht hat. Bevor Sie einen Hint setzen, prüfen Sie, ob aktualisierte Statistiken, ein Histogramm oder ein besserer Index das Problem lösen. Hints veralten, wenn sich Datenverteilungen ändern, und niemand prüft sie nach.

7.7 Optimizer Trace

```
SET optimizer_trace = 'enabled=on';
SELECT ... ;    -- die Abfrage
SELECT * FROM information_schema.OPTIMIZER_TRACE\G
SET optimizer_trace = 'enabled=off';
```

Der Trace zeigt jede erwogene Alternative mit ihren geschätzten Kosten. Damit lässt sich beantworten, warum der Optimizer einen scheinbar perfekten Index nicht nimmt: Meist hält er den Bereich für zu groß oder die Kosten für Zeilenzugriffe für zu hoch.

7.8 Typische Optimierungsschritte in Reihenfolge

1. Langsame Abfrage identifizieren (Slow Log, Kapitel 9). Nicht raten.
2. `ANALYZE FORMAT=JSON` ausführen. Den Schritt mit der höchsten `r_total_time_ms` finden.
3. Ist dort `type=ALL` auf einer großen Tabelle? Index für die `WHERE`- und `JOIN`-Spalten anlegen.
4. Ist `rows` gegen `r_rows` stark überschätzt? `ANALYZE TABLE ... PERSISTENT FOR ALL`.
5. Steht `Using filesort` oder `Using temporary` bei großen Mengen? Index an `ORDER BY` oder `GROUP BY` anpassen.
6. Abfrage umformulieren: Unterabfragen zu Joins, Funktionen aus `WHERE` entfernen, `OR` zu `UNION`.
7. Covering Index prüfen, wenn die Abfrage sehr häufig läuft.
8. Erst jetzt: Serverparameter, Hardware, Caching in der Anwendung.

8. Server-Tuning STUFE 3

Es gibt etwa 600 Servervariablen. Ein Dutzend davon entscheidet über 90 Prozent der Performance. Der Rest ist Feinjustierung für Spezialfälle. Ändern Sie immer nur einen Parameter, messen Sie, dokumentieren Sie. Konfiguration liegt in `/etc/mysql/mariadb.conf.d/50-server.cnf` (Debian/Ubuntu) bzw. `/etc/my.cnf.d/server.cnf` (RHEL) im Abschnitt `[mysqld]`.

8.1 Die Parameter, die zählen

PARAMETER	AUSGANGSWERT	ERLÄUTERUNG
<code>innodb_buffer_pool_size</code>	50 bis 70 % des RAM auf einem dedizierten Server	Der wichtigste Wert. Ziel: das gesamte Working Set (aktive Daten plus Indizes) passt hinein. Zur Laufzeit änderbar. Bei gemischten Servern (Webserver auf derselben Maschine) entsprechend weniger.
<code>innodb_log_file_size</code>	1 bis 4 GB, so dass eine Stunde Schreiblast hineinpasst	Messen: <code>Innodb_os_log_written</code> über eine Stunde. Zu klein: Checkpoint-Stalls. Ab 10.9 online änderbar.
<code>innodb_flush_log_at_trx_commit</code>	1 (sicher) oder 2 (schnell)	1: <code>fsync</code> bei jedem Commit, volle Durability. 2: Schreiben ins OS bei Commit, <code>fsync</code> einmal pro Sekunde; bei OS-Absturz bis zu einer Sekunde Datenverlust, bei MariaDB-Absturz kein Verlust. 0: nur einmal pro Sekunde, auch bei MariaDB-Absturz Verlust. Für Buchhaltungsdaten 1, für Logtabellen oder Replikate 2.
<code>innodb_flush_method</code>	<code>O_DIRECT</code>	Umgeht den OS Page Cache, verhindert doppeltes Caching. Auf Linux Standardempfehlung. Ab 10.11 auf Linux automatisch bevorzugt.
<code>innodb_io_capacity/_max</code>	SSD: 2000 / 4000. NVMe: höher. HDD: 200 / 400	Steuert, wie aggressiv Hintergrund-Flushing arbeitet. Zu niedrig: dirty Pages stauen sich. Zu hoch: unnötige Schreiblast, verkürzte SSD-Lebensdauer.
<code>innodb_buffer_pool_instances</code>	Entfällt ab 10.5	Bis 10.4: 8 bei großem Pool. Ab 10.5 gibt es nur noch eine Instanz mit verbesserter Synchronisation.
<code>innodb_read_io_threads/write</code>	8 / 8 bei SSD	Standard 4. Bei NVMe darf man höher gehen.

<code>max_connections</code>	200 bis 500	Nicht 5000. Mehr gleichzeitig aktive Threads als CPU-Kerne mal zwei bringen nichts, sie konkurrieren nur. Verbindungspooling in der Anwendung oder über MaxScale/ProxySQL ist die Lösung, nicht ein höherer Wert.
<code>thread_handling</code>	<code>pool-of-threads</code>	Der MariaDB-Threadpool (in der Community-Version enthalten, in MySQL nur Enterprise) begrenzt gleichzeitig laufende Threads auf die CPU-Anzahl und stabilisiert den Durchsatz unter Last erheblich.
<code>thread_cache_size</code>	Automatisch (Standard 256)	Bei <code>thread_handling=one-thread-per-connection</code> relevant, sonst nicht.
<code>tmp_table_size / max_heap_table_size</code>	64M bis 256M, beide gleich	Obergrenze für temporäre Tabellen im Speicher. Größere werden auf Platte (Aria) ausgelagert. <code>Created_tmp_disk_tables</code> im Verhältnis zu <code>Created_tmp_tables</code> beobachten. TEXT/BLOB-Spalten erzwingen immer Platte.
<code>sort_buffer_size</code>	2M bis 4M	Je Verbindung, nicht global. Große Werte helfen selten und kosten Speicher. Statt zu erhöhen: Index für ORDER BY.
<code>join_buffer_size</code>	1M bis 4M	Nur relevant bei Joins ohne Index. Wenn dieser Wert etwas bringt, fehlt ein Index.
<code>table_open_cache</code>	4000 oder mehr	<code>Opened_tables</code> sollte im Betrieb nicht steigen. Zusammen mit <code>open_files_limit</code> und dem OS-Limit <code>ulimit -n</code> setzen.
<code>table_definition_cache</code>	Anzahl Tabellen plus Puffer	Bei tausenden Tabellen (Multi-Tenant) wichtig.
<code>query_cache_type</code>	0 (aus)	Der Query Cache ist in MariaDB noch vorhanden, aber ein globaler Mutex, der unter Last mehr schadet als nützt. In MySQL 8 entfernt. Ausschalten, Caching in die Anwendungsschicht verlagern.
<code>innodb_stats_persistent</code>	ON	Statistiken überleben Neustarts, Pläne bleiben stabil.

<code>innodb_autoinc_lock_mode</code>	2	Interleaved, höchster Durchsatz bei parallelen Inserts. Setzt ROW-Binlog voraus.
<code>innodb_lock_wait_timeout</code>	5 bis 15	Statt 50. Frühes Scheitern ist besser als langes Warten.
<code>transaction_isolation</code>	REPEATABLE-READ oder READ-COMMITTED	Siehe Kapitel 5.
<code>character_set_server / collation_server</code>	utf8mb4 / utf8mb4_unicode_ci	Verhindert Zeichensatzchaos bei neuen Tabellen.
<code>sql_mode</code>	STRICT_TRANS_TABLES, ERROR_FOR_DIVISION_BY_ZERO, NO_ENGINE_SUBSTITUTION, ONLY_FULL_GROUP_BY	Strikter Modus: Ungültige Daten erzeugen Fehler statt stiller Kürzung. Standard seit 10.2.4.

8.2 Beispielkonfiguration für einen dedizierten Server mit 64 GB RAM und NVMe

```
[mysqld]
# Speicher
innodb_buffer_pool_size      = 44G
innodb_log_buffer_size      = 64M
tmp_table_size               = 128M
max_heap_table_size          = 128M
sort_buffer_size              = 4M
join_buffer_size             = 4M

# Redo und Durability
innodb_log_file_size         = 4G
innodb_flush_log_at_trx_commit = 1
innodb_flush_method          = O_DIRECT
innodb_doublewrite           = ON

# I/O
innodb_io_capacity           = 4000
innodb_io_capacity_max       = 8000
innodb_read_io_threads       = 8
innodb_write_io_threads      = 8

# Verbindungen
max_connections               = 400
thread_handling                = pool-of-threads
thread_pool_size               = 16          # etwa Anzahl CPU-Kerne
innodb_lock_wait_timeout      = 10
wait_timeout                   = 600
interactive_timeout            = 600

# Tabellen und Caches
table_open_cache               = 8000
table_definition_cache         = 4000
open_files_limit               = 65535
query_cache_type               = 0
query_cache_size               = 0

# Binlog und Replikation
log_bin                        = /var/lib/mysql/binlog
binlog_format                  = ROW
binlog_expire_logs_seconds     = 604800      # 7 Tage
sync_binlog                    = 1
server_id                      = 1
gtid_strict_mode               = ON

# Logging
slow_query_log                 = 1
slow_query_log_file            = /var/log/mysql/slow.log
long_query_time                 = 0.5
log_slow_verbosity              = query_plan,explain
log_queries_not_using_indexes   = 0          # sonst zu viel Rauschen

# Sicherheit und Verhalten
character_set_server            = utf8mb4
collation_server                = utf8mb4_unicode_ci
sql_mode                       =
STRICT_TRANS_TABLES,ERROR_FOR_DIVISION_BY_ZERO,NO_ENGINE_SUBSTITUTION,ONLY_FULL_GROUP_BY
local_infile                    = 0
```

```
skip_name_resolve      = 1
performance_schema     = ON
```

WARNUNG

Diese Werte sind ein Ausgangspunkt für genau die beschriebene Hardware. Ein Server mit 8 GB RAM, der auch einen Webserver beherbergt, braucht andere Werte (Buffer Pool vielleicht 3 GB). Prüfen Sie nach jedem Neustart `SHOW GLOBAL VARIABLES`, ob die Werte übernommen wurden, und beobachten Sie den Speicherverbrauch des Prozesses eine Woche lang. Ein Server im Swap ist langsamer als jeder falsche Parameter.

8.3 Betriebssystem und Hardware

- **Speichermedium:** NVMe-SSD ist Standard. fsync-Latenz ist die entscheidende Größe für Commit-Durchsatz; Enterprise-SSDs mit Power-Loss-Protection liefern konstant niedrige Latenz. Netzwerkspeicher und Cloud-Volumes haben höhere Latenz; dort ist `innodb_flush_log_at_trx_commit=2` oft die einzige Möglichkeit, akzeptablen Schreibdurchsatz zu erreichen.
- **Dateisystem:** XFS oder ext4 mit `noatime`. ZFS ist möglich (Kompression, Snapshots), erfordert aber abgestimmte Einstellungen (`recordsize 16K` für Daten, `innodb_doublewrite=OFF` möglich, `innodb_flush_method=fsync`).
- **Swappiness:** `vm.swappiness=1`. Der Server darf nicht swappen; besser `innodb_buffer_pool_size` reduzieren.
- **Transparent Huge Pages:** auf `never` setzen, sie verursachen Latenzspitzen.
- **I/O-Scheduler:** `none` oder `mq-deadline` für NVMe.
- **CPU:** Wenige schnelle Kerne schlagen viele langsame für OLTP, weil eine einzelne Abfrage nur einen Thread nutzt. CPU-Frequenzskalierung auf `performance`.
- **Netzwerk:** Latenz zwischen Anwendung und Datenbank zählt bei jedem Roundtrip. 0,5 ms mal 200 Abfragen je Seitenaufruf sind 100 ms. Anwendung und Datenbank ins selbe Rechenzentrum, idealerweise dasselbe Rack.
- **Virtualisierung und Container:** Funktioniert, aber I/O-Latenz und CPU-Steal messen. In Kubernetes Speicher-Limits mit Buffer Pool abstimmen, sonst wird der Prozess vom OOM-Killer beendet.

8.4 Was kein Tuning löst

Kein Parameter kompensiert einen fehlenden Index, einen fehlenden Primärschlüssel, eine N+1-Abfragestruktur oder eine Tabelle mit 40 Spalten, von denen 30 TEXT sind. Serverparameter sind der letzte Schritt, nicht der erste. Die Reihenfolge lautet: Schema, Indizes, Abfragen, Anwendungsarchitektur (Caching, Pooling), dann Server, dann Hardware.

9. Monitoring und Diagnose STUFE 3

9.1 Das Slow Query Log

Das wichtigste Werkzeug zum Finden teurer Abfragen. Mit `long_query_time = 0.5` wird alles protokolliert, was länger als eine halbe Sekunde dauert. Für eine Bestandsaufnahme kann man den Wert vorübergehend auf 0 setzen und wenige Minuten alles mitschreiben (Vorsicht bei hoher Last: Logvolumen). Das Rohlog ist unhandlich; `pt-query-digest` aus dem Percona Toolkit aggregiert es nach Abfragemuster und zeigt, welche Abfrageklasse die meiste Gesamtzeit verbraucht. Das ist die Kennzahl, die zählt: Eine Abfrage mit 20 ms, die 10.000-mal pro Minute läuft, kostet mehr als eine mit 5 Sekunden, die einmal pro Stunde läuft.

```
SET GLOBAL slow_query_log = ON;
SET GLOBAL long_query_time = 0.2;
SET GLOBAL log_slow_verbosity = 'query_plan,explain';
-- Auswertung:
pt-query-digest /var/log/mysql/slow.log --limit 20 > digest.txt
```

9.2 Statusvariablen

VARIABLE	BEDEUTUNG	BEWERTUNG
Threads_running	Gleichzeitig aktive Abfragen	Dauerhaft über CPU-Kernzahl mal zwei: Überlast
Threads_connected / max_used_connections	Offene Verbindungen und Spitzenwert	Nahe max_connections: Pooling prüfen
Innodb_buffer_pool_reads vs. _read_requests	Plattenlesen vs. Gesamtlesen	Trefferquote über 99 % anstreben
Innodb_buffer_pool_wait_free	Wartezeiten auf freie Seiten	Größer 0 dauerhaft: Pool zu klein oder Flushing zu langsam
Innodb_row_lock_waits / _time_avg	Sperrwartezeiten	Steigend: Sperrkonflikte, Transaktionen verkürzen
Innodb_deadlocks	Deadlocks seit Start	Rate beobachten, nicht Absolutwert
Created_tmp_disk_tables / Created_tmp_tables	Temporäre Tabellen auf Platte	Über 10 bis 20 % Anteil: tmp_table_size oder Abfragen prüfen
Select_full_join	Joins ohne Index	Sollte nahe 0 sein
Select_scan	Vollständige Tabellenscans	Im Verhältnis zu Com_select bewerten
Sort_merge_passes	Sortierungen, die nicht in sort_buffer passten	Steigend: Indizes für ORDER BY
Handler_read_rnd_next	Zeilen sequentiell gelesen	Hoch im Verhältnis zu Handler_read_key: Scans dominieren

Opened_tables	Tabellenöffnungen	Steigt im Betrieb: table_open_cache zu klein
Aborted_connects / Aborted_clients	Fehlgeschlagene und abgebrochene Verbindungen	Steigend: Netzwerk, Timeouts, Passwörter
Binlog_commits / Binlog_group_commits	Group-Commit-Effizienz	Verhältnis zeigt, wie viele Commits je fsync gebündelt werden

```

SHOW GLOBAL STATUS;                -- alles, Deltas über Zeit bilden
SHOW GLOBAL STATUS LIKE 'Innodb_%';
SHOW ENGINE INNODB STATUS\G        -- Semaphoren, Deadlocks, Buffer Pool, Row
Operations, History List
SHOW FULL PROCESSLIST;              -- was läuft gerade, State-Spalte beachten
SELECT * FROM information_schema.PROCESSLIST WHERE COMMAND != 'Sleep' ORDER BY TIME
DESC;
```

9.3 Performance Schema und sys-Schema

Das Performance Schema instrumentiert den Server auf Ebene von Statements, Waits, Sperren und I/O. Es kostet einige Prozent Leistung und ist in MariaDB standardmäßig ausgeschaltet; auf produktiven Systemen, die man verstehen will, gehört es an. Das sys-Schema (ab 10.6 enthalten) stellt lesbare Views darüber.

```

-- Top-Abfragen nach Gesamtlatenz
SELECT * FROM sys.statement_analysis ORDER BY total_latency DESC LIMIT 10;
-- Abfragen mit Tabellenscans
SELECT * FROM sys.statements_with_full_table_scans LIMIT 10;
-- Ungenutzte und redundante Indizes
SELECT * FROM sys.schema_unused_indexes;
SELECT * FROM sys.schema_redundant_indexes;
-- Tabellen nach I/O
SELECT * FROM sys.schema_table_statistics ORDER BY total_latency DESC LIMIT 10;
-- Wer wartet gerade auf wen
SELECT * FROM sys.innodb_lock_waits;
-- Dateisystem-Wartezeiten
SELECT * FROM sys.io_global_by_wait_by_latency;
```

Ohne Performance Schema liefern information_schema.TABLE_STATISTICS, INDEX_STATISTICS und USER_STATISTICS (aktivieren mit userstat=ON) ähnliche, günstigere Zähler auf Tabellenebene.

9.4 Werkzeuge

WERKZEUG	ZWECK
Percona Monitoring and Management (PMM)	Kostenlose Komplettlösung: Grafana-Dashboards, Query Analytics, Historie. Empfehlung für jeden ernsthaften Betrieb.
Prometheus mysqld_exporter + Grafana	Baukasten, wenn PMM zu schwer ist
Percona Toolkit	pt-query-digest (Slow Log), pt-online-schema-change (Online-DDL), pt-table-checksum (Replikationskonsistenz), pt-kill, pt-duplicate-key-checker
mytop, innotop	Live-Ansicht der Prozessliste und InnoDB-Kennzahlen

MariaDB Enterprise Monitor Kommerzielle Angebote von MariaDB plc
/ SkySQL

mysqltuner.pl

Schneller Konfigurationscheck; Empfehlungen kritisch lesen, nicht blind übernehmen

9.5 Diagnose-Ablauf bei "die Datenbank ist langsam"

1. `SHOW FULL PROCESSLIST`: Wie viele laufen, was tun sie, in welchem State? Viele in `Waiting for table metadata lock`: eine DDL oder eine offene Transaktion blockiert. Viele in `Sending data` oder `Copying to tmp table`: teure Abfragen.
2. OS-Ebene: `top`, `iostat -x 1`, `vmstat 1`. CPU-gebunden, I/O-gebunden (hohe %util, hohe await) oder Swap?
3. `SHOW ENGINE INNODB STATUS`: History List, Semaphore Waits, Pending Reads/Writes, Log Sequence Number vs. Checkpoint (Abstand zeigt Checkpoint-Druck).
4. Slow Log der letzten Minuten aggregieren. Welches Muster dominiert?
5. Hat sich etwas geändert? Deployment, Datenmenge, Statistik-Neuberechnung, Cron-Job, Backup?
6. Kurzfristig: blockierende Transaktion beenden (`KILL <id>`), schädliche Abfrage killen. Langfristig: Ursache aus Kapitel 6 und 7 beseitigen.

10. Skalierung und Hochverfügbarkeit

STUFE 3 BIS 4

Zwei verschiedene Ziele werden oft vermischt: Skalierung (mehr Last verarbeiten) und Hochverfügbarkeit (Ausfälle überstehen). Replikation dient beiden, aber in unterschiedlicher Konfiguration.

10.1 Vertikal zuerst

Ein einzelner MariaDB-Server auf moderner Hardware verarbeitet zehntausende Transaktionen pro Sekunde und Datenbanken im Terabyte-Bereich. Vertikale Skalierung (mehr RAM, schnellere NVMe, mehr Kerne) ist fast immer günstiger und risikoärmer als verteilte Architekturen, weil sie keine Anwendungsänderung erfordert. Erst wenn die größte wirtschaftlich sinnvolle Maschine nicht reicht oder Verfügbarkeitsanforderungen es erzwingen, wird verteilt.

10.2 Asynchrone Replikation

Der Primary schreibt das Binlog, das Replikat holt es per I/O-Thread, speichert es im Relay Log und wendet es per SQL-Thread an. Der Primary wartet nicht auf das Replikat; deshalb "asynchron". Das Replikat kann Sekunden bis Minuten hinterherhängen (Replication Lag).

```
-- Primary: server_id, log_bin, binlog_format=ROW, gtid_strict_mode=ON
CREATE USER 'repl'@'%' IDENTIFIED BY '...';
GRANT REPLICATION SLAVE ON *.* TO 'repl'@'%';

-- Replikat (nach Einspielen eines konsistenten Backups mit bekannter GTID-Position)
SET GLOBAL gtid_slave_pos = '0-1-123456';
CHANGE MASTER TO MASTER_HOST='primary', MASTER_USER='repl', MASTER_PASSWORD='...',
                  MASTER_USE_GTID = slave_pos, MASTER_SSL = 1;
START REPLICA;           -- START SLAVE in älteren Versionen
SHOW REPLICA STATUS\G    -- Seconds_Behind_Master, Slave_IO_Running,
Slave_SQL_Running, Last_Error
```

MariaDB-GTIDs (Global Transaction ID) haben das Format Domain-ServerID-Sequenz und sind nicht kompatibel mit MySQL-GTIDs. Sie erlauben Failover ohne manuelle Positionsbestimmung. `gtid_strict_mode` verhindert, dass ein Replikat aus der Reihe fällt.

Parallele Replikation: `slave_parallel_threads = 8` und `slave_parallel_mode = optimistic` lassen das Replikat mehrere Transaktionen gleichzeitig anwenden. Ohne Parallelisierung ist ein einzelner SQL-Thread auf dem Replikat häufig der Engpass, während der Primary mit hundert Threads schreibt.

Semi-synchrone Replikation: Der Primary wartet beim Commit, bis mindestens ein Replikat den Binlog-Eintrag empfangen (nicht angewendet) hat. Bei Ausfall des Primary ist garantiert keine bestätigte Transaktion verloren. Kostet eine Netzwerk-Roundtrip-Latenz pro Commit. Für Failover-Szenarien mit Datenverlustverbot die richtige Wahl (`rpl_semi_sync_master_enabled`, `rpl_semi_sync_slave_enabled`).

10.3 Lesereplikate und Read/Write-Split

Der klassische Skalierungspfad: Alle Schreibvorgänge zum Primary, Lesevorgänge auf ein oder mehrere Replikate verteilen. Reporting und Backups laufen auf einem Replikat und stören den Betrieb nicht. Die Anwendung muss den Lag verkraften: Ein Nutzer, der gerade gespeichert hat und sofort vom Replikat liest, sieht seine Änderung eventuell nicht. Lösungen: Lesen nach eigenem Schreiben vom Primary, oder `MASTER_GTID_WAIT()` auf dem Replikat.

10.4 Galera Cluster: synchrone Multi-Primary-Replikation

Galera (in MariaDB als MariaDB Galera Cluster integriert) repliziert Transaktionen synchron zu allen Knoten mittels zertifikationsbasierter Replikation: Beim Commit wird das Writeset an alle Knoten gesendet, jeder prüft auf Konflikte mit laufenden Transaktionen, dann wird überall gleichzeitig bestätigt. Alle Knoten können schreiben, es gibt keinen Lag, ein Knotenausfall ist transparent.

VORTEIL	NACHTEIL
Kein Datenverlust bei Knotenausfall, keine Failover-Logik	Commit-Latenz = Netzwerk-Roundtrip zum langsamsten Knoten; ungeeignet über weite Distanzen
Jeder Knoten liest konsistent	Schreibdurchsatz skaliert nicht; jeder Knoten wendet jedes Writeset an
Automatischer Beitritt neuer Knoten (SST/IST)	Große Transaktionen (Massen-UPDATE) sind problematisch; Zertifikationskonflikte bei Schreibvorgängen auf dieselben Zeilen von verschiedenen Knoten
Quorum verhindert Split Brain	Mindestens drei Knoten (oder zwei plus Arbitrator), alle Tabellen brauchen Primärschlüssel und InnoDB

Praxisregel: Galera mit drei Knoten im selben Rechenzentrum, aber alle Schreibvorgänge über MaxScale auf einen Knoten leiten (Single-Writer). Das vermeidet Konflikte und nutzt trotzdem die sofortige Failover-Fähigkeit.

10.5 MaxScale: der intelligente Proxy

MariaDB MaxScale sitzt zwischen Anwendung und Datenbankservern. Er erkennt Primary und Replikate automatisch, trennt Lese- und Schreibvorgänge auf Statement-Ebene (readwritesplit-Router), führt automatisches Failover mit GTID-basierter Neuausrichtung der Replikate durch (mariadbmon), begrenzt Verbindungen, kann Abfragen filtern und maskieren. Für die Anwendung ist er ein einzelner MariaDB-Endpunkt. Lizenzhinweis: MaxScale steht unter der Business Source License; Produktivnutzung mit mehr als zwei Datenbankservern erfordert eine Lizenz von MariaDB plc. Alternativen: ProxySQL (GPL), HAProxy (nur TCP, kein SQL-Verständnis), Orchestrator für Failover.

10.6 Partitionierung

Partitionierung teilt eine Tabelle intern in mehrere physische Teile, die nach außen als eine Tabelle erscheinen. Nutzen: Partition Pruning (Abfragen mit Bedingung auf dem Partitionsschlüssel lesen nur betroffene Partitionen), schnelles Löschen alter Daten durch DROP PARTITION statt DELETE, und Wartung je Partition.

```
CREATE TABLE zugriffsprotokoll (
  id BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
  zeitpunkt DATETIME NOT NULL,
  ...,
  PRIMARY KEY (id, zeitpunkt)          -- Partitionsschlüssel muss Teil jedes UNIQUE-
Keys sein
) PARTITION BY RANGE (TO_DAYS(zeitpunkt)) (
  PARTITION p2026_08 VALUES LESS THAN (TO_DAYS('2026-09-01')),
  PARTITION p2026_09 VALUES LESS THAN (TO_DAYS('2026-10-01')),
  PARTITION pmax      VALUES LESS THAN MAXVALUE
);
ALTER TABLE zugriffsprotokoll DROP PARTITION p2026_08;  -- Sekundenbruchteile statt
Stunden DELETE
```

Partitionierung ist kein Performance-Allheilmittel. Abfragen ohne Bedingung auf dem Partitionsschlüssel müssen alle Partitionen durchsuchen und werden langsamer, nicht schneller. Fremdschlüssel auf partitionierte Tabellen sind nicht möglich. Sinnvoll für Zeitreihen und Protokolldaten mit Aufbewahrungsfristen, selten für Stammdaten.

10.7 Sharding

Sharding verteilt Daten nach einem Schlüssel (Mandant, Region, Kundennummer) auf getrennte Datenbankserver. Es ist die einzige Methode, Schreibdurchsatz jenseits eines Servers zu skalieren, und die teuerste: Joins über Shards sind unmöglich, Transaktionen über Shards erfordern Zwei-Phasen-Commit, Rebalancing ist komplex, jede Abfrage braucht den Shard-Schlüssel. Bevor Sie shardieren: Archivieren Sie kalte Daten, prüfen Sie MyRocks oder ColumnStore für die großen Tabellen, lagern Sie Ereignisdaten in ein dafür gebautes System aus. Wenn Sharding unvermeidlich ist, ist der Mandant in Multi-Tenant-Systemen der natürliche Schlüssel: eine Datenbank je Mandant auf verschiedenen Servern, Verzeichnisdienst in der Anwendung.

10.8 Referenzarchitekturen

ANFORDERUNG	ARCHITEKTUR	RPO / RTO
Kleines System, Ausfall von Stunden tolerierbar	Ein Server, tägliches Backup plus Binlog-Sicherung	Minuten / Stunden
Mittleres System, Ausfall von Minuten tolerierbar	Primary plus ein Replikat (semi-sync), MaxScale oder Orchestrator für Failover, Backup vom Replikat	0 bis Sekunden / Minuten
Hohe Verfügbarkeit, kein Datenverlust	Galera drei Knoten, MaxScale davor, zusätzliches asynchrones Replikat an zweitem Standort für Disaster Recovery und Backup	0 / Sekunden
Hohe Leselast	Primary plus mehrere Replikate, Read/Write-Split, Anwendungscache (Redis) davor	je nach Basis

11. Backup und Wiederherstellung

STUFE 3

Ein Backup, dessen Wiederherstellung nie getestet wurde, ist eine Hoffnung, kein Backup. Die relevanten Größen sind RPO (Recovery Point Objective: wie viel Datenverlust ist akzeptabel) und RTO (Recovery Time Objective: wie lange darf die Wiederherstellung dauern). Beide bestimmen die Methode.

11.1 Logisches Backup: mariadb-dump

```
mariadb-dump --single-transaction --routines --triggers --events \
--master-data=2 --flush-logs --hex-blob --all-databases | gzip > voll_$(date
+%F).sql.gz
# --single-transaction: konsistenter Snapshot für InnoDB ohne Sperren
# --master-data=2: Binlog-Position als Kommentar für PITR und Replikatsaufbau
# Wiederherstellung:
gunzip < voll.sql.gz | mariadb
```

Vorteile: portabel, lesbar, selektiv, funktioniert über Versionen und Engines hinweg. Nachteile: Wiederherstellung ist langsam, weil jede Zeile per INSERT eingefügt und jeder Index neu aufgebaut wird. Bei 100 GB Daten sind das Stunden. Für Datenbanken bis etwa 20 bis 50 GB oder für Schema- und Einzeltabellensicherungen geeignet. mydumper/myloader parallelisieren und sind erheblich schneller.

11.2 Physisches Backup: Mariabackup

Mariabackup (Fork von Percona XtraBackup, in MariaDB enthalten) kopiert die InnoDB-Dateien im laufenden Betrieb und zeichnet parallel das Redo Log auf. Anschließend wird das Backup "vorbereitet" (Redo Log angewendet), sodass es einen konsistenten Zustand darstellt. Wiederherstellung ist ein Kopiervorgang: so schnell wie die Platte. Inkrementelle Backups sichern nur geänderte Seiten.

```
mariabackup --backup --target-dir=/backup/voll --user=backup --password=... --parallel=4
mariabackup --prepare --target-dir=/backup/voll
# Inkrementell:
mariabackup --backup --target-dir=/backup/inc1 --incremental-basedir=/backup/voll ...
mariabackup --prepare --target-dir=/backup/voll --incremental-dir=/backup/inc1
# Wiederherstellung (Server gestoppt, datadir leer):
mariabackup --copy-back --target-dir=/backup/voll
chown -R mysql:mysql /var/lib/mysql
```

Die Datei xtrabackup_binlog_info im Backup enthält Binlog-Position und GTID für Point-in-Time-Recovery und Replikatsaufbau.

11.3 Point-in-Time-Recovery

Ein Vollbackup stellt den Stand von 02:00 Uhr wieder her. Um 14:37 löscht jemand versehentlich eine Tabelle. Mit dem Binlog lässt sich der Stand von 14:36 rekonstruieren: Vollbackup einspielen, dann alle Binlog-Ereignisse vom Backup-Zeitpunkt bis kurz vor dem Fehler nachspielen.

```

mariadb-binlog --start-position=1234 binlog.000042 \
  --stop-datetime="2026-09-06 14:36:59" binlog.000042 binlog.000043 | mariadb
# Alternativ die schädliche Transaktion per GTID überspringen:
mariadb-binlog --start-position=1234 --stop-position=98765 ... | mariadb # bis vor
DROP
mariadb-binlog --start-position=98900 ... | mariadb # ab nach DROP

```

Voraussetzung: Binlogs werden mindestens so lange aufbewahrt wie der Abstand zwischen zwei Vollbackups, idealerweise länger, und sie werden vom Server weg gesichert (z. B. stündlicher Rsync oder mariadb-binlog - read-from-remote-server --raw als Live-Kopie).

11.4 Strategie

ELEMENT	EMPFEHLUNG
Vollbackup	Täglich mit Mariabackup vom Replikat, nachts
Inkrementell	Stündlich, falls RPO es erfordert; sonst Binlog-Sicherung reicht
Binlog	Kontinuierlich auf getrenntes System kopieren, Aufbewahrung mindestens 7 Tage
Logischer Dump	Wöchentlich zusätzlich, als versionsunabhängige Notfallreserve
Aufbewahrung	7 Tage täglich, 4 Wochen wöchentlich, 12 Monate monatlich; gesetzliche Anforderungen (HGB, AO) beachten, aber nicht mit dem Datenbank-Backup abbilden, sondern mit Archivexporten
Speicherort	Anderer physischer Server, anderer Brandabschnitt oder Region, verschlüsselt, mit Integritätsprüfung (Prüfsumme)
Test	Monatlich vollständige Wiederherstellung auf Testsystem, Dauer messen, Ergebnis protokollieren. Quartalsweise PITR-Übung.
Überwachung	Alarm, wenn Backup fehlt, zu klein ist oder Prüfsumme abweicht

WAS EIN REPLIKAT NICHT IST

Ein Replikat ist kein Backup. Ein DROP TABLE auf dem Primary ist Millisekunden später auch auf dem Replikat ausgeführt. Ein Replikat schützt vor Hardwareausfall, nicht vor menschlichen Fehlern, fehlerhaften Deployments oder Ransomware. Verzögerte Replikation (MASTER_DELAY = 3600) ist ein zusätzlicher Puffer, ersetzt aber kein Backup außerhalb der Reichweite des Datenbankservers.

12. Sicherheit und Compliance STUFE 3

12.1 Benutzer, Rechte, Rollen

Jede Anwendung erhält einen eigenen Benutzer mit genau den Rechten, die sie braucht, beschränkt auf ihre Datenbank und, wo möglich, ihre Herkunftsadresse. root wird nie von Anwendungen verwendet. Rollen (seit 10.0) bündeln Rechte und machen die Verwaltung übersichtlich.

```
CREATE ROLE app_lesen;
GRANT SELECT ON hv.* TO app_lesen;
CREATE ROLE app_schreiben;
GRANT SELECT, INSERT, UPDATE, DELETE ON hv.* TO app_schreiben;

CREATE USER 'hv_app'@'10.0.1.%' IDENTIFIED BY '...' PASSWORD EXPIRE INTERVAL 180 DAY;
GRANT app_schreiben TO 'hv_app'@'10.0.1.%';
SET DEFAULT ROLE app_schreiben FOR 'hv_app'@'10.0.1.%';

CREATE USER 'report'@'10.0.2.%' IDENTIFIED BY '...' WITH MAX_QUERIES_PER_HOUR 5000;
GRANT app_lesen TO 'report'@'10.0.2.%';
SET DEFAULT ROLE app_lesen FOR 'report'@'10.0.2.%';

CREATE USER 'backup'@'localhost' IDENTIFIED BY '...';
GRANT RELOAD, PROCESS, LOCK TABLES, BINLOG MONITOR ON *.* TO 'backup'@'localhost';

SHOW GRANTS FOR 'hv_app'@'10.0.1.%';
SELECT user, host, plugin FROM mysql.user;
```

Authentifizierung: ed25519-Plugin statt mysql_native_password für Passwörter, unix_socket für lokale Systembenutzer ohne Passwort, PAM oder LDAP für Verzeichnisintegration. Passwortrichtlinien über simple_password_check-Plugin. Kontosperrung nach Fehlversuchen mit max_password_errors.

12.2 Netzwerk und Transport

- bind_address auf die interne Adresse, nie 0.0.0.0 ohne Firewall. Port 3306 nur aus dem Anwendungsnetz erreichbar.
- TLS für alle Verbindungen: ssl_cert, ssl_key, ssl_ca setzen, require_secure_transport = ON erzwingt es. Benutzer mit REQUIRE SSL oder REQUIRE X509 anlegen.
- skip_name_resolve = 1: keine DNS-Abfragen bei Verbindungsaufbau, keine Host-Namen in Grants (nur IP).
- local_infile = 0: verhindert, dass ein kompromittierter Server Dateien vom Client liest.
- secure_file_priv auf ein Verzeichnis beschränken oder leer setzen, um LOAD DATA und SELECT ... INTO OUTFILE zu kontrollieren.

12.3 Verschlüsselung ruhender Daten

MariaDB bietet transparente Datenverschlüsselung (Data-at-Rest Encryption) für InnoDB-Tablespaces, Redo Log, Binlog und temporäre Dateien. Schlüssel liefert ein Plugin: file_key_management (Schlüssel in Datei, für Tests), AWS KMS, HashiCorp Vault. Die Verschlüsselung ist für Anwendungen unsichtbar und kostet wenige Prozent CPU.

```
[mysqld]
plugin_load_add          = file_key_management
file_key_management_filename = /etc/mysql/keys.enc
file_key_management_filekey = FILE:/etc/mysql/.keyfile
innodb_encrypt_tables    = FORCE
innodb_encrypt_log       = ON
innodb_encrypt_temporary_tables = ON
encrypt_binlog           = ON
encrypt_tmp_files        = ON
aria_encrypt_tables      = ON
```

Zusätzlich: Backups verschlüsseln (Mariabackup unterstützt das nicht direkt; Ausgabe durch gpg oder age leiten), Festplattenverschlüsselung auf OS-Ebene als zweite Schicht.

12.4 Audit

Das MariaDB Audit Plugin (`server_audit`) protokolliert Verbindungen, Abfragen und Tabellenzugriffe wahlweise für alle oder ausgewählte Benutzer in eine Datei oder Syslog. Für den Nachweis, wer wann welche personenbezogenen Daten abgefragt hat (Art. 30, 32 DSGVO Rechenschaftspflicht), ist das die technische Grundlage. Abfrageprotokollierung auf produktiven Systemen mit hoher Last erzeugt große Datenmengen; meist genügt `server_audit_events = CONNECT, TABLE` mit Ausschluss der Anwendungsbenutzer und vollständigem Protokoll für administrative Konten.

12.5 SQL-Injection

Die häufigste und am besten verstandene Angriffsart auf Datenbanken. Sie entsteht ausschließlich dadurch, dass Nutzereingaben in SQL-Text eingefügt werden. Die einzige vollständige Abwehr sind parametrisierte Abfragen (Prepared Statements), bei denen Daten nie Teil des SQL-Textes werden. Jede Programmiersprache und jedes Framework unterstützt das. Escaping ist Notbehelf, Whitelisting für Spaltennamen und Sortierrichtungen (die nicht parametrisierbar sind) Pflicht. Zusätzliche Schichten: minimale Rechte des Anwendungsbenutzers (kein DROP, kein FILE, kein SUPER), keine Fehlermeldungen der Datenbank an Endnutzer, Web Application Firewall.

12.6 DSGVO-relevante Maßnahmen

ANFORDERUNG	TECHNISCHE UMSETZUNG
Zugriffskontrolle	Rollen, Least Privilege, personalisierte Admin-Konten, keine Sammelkonten
Verschlüsselung	TLS, Data-at-Rest, verschlüsselte Backups
Protokollierung	Audit Plugin für Zugriffe auf personenbezogene Tabellen
Löschkonzept	Löschroutinen per Event oder Job, Partitionierung nach Aufbewahrungsfrist, Prüfung, dass Löschungen auch in Backups nach Ablauf der Aufbewahrungsfrist wirksam werden
Auskunft	Abfragen und Exporte je betroffener Person vorbereiten, personenbezogene Spalten dokumentieren
Pseudonymisierung in Testsystemen	Produktivdaten nie unverändert in Test oder Entwicklung; Maskierung per Skript oder MaxScale-Masking-Filter
Verfügbarkeit (Art. 32)	Backup, Wiederherstellungstests, Replikation

Die rechtliche Bewertung von Aufbewahrungsfristen und Löschpflichten gehört zum Datenschutzbeauftragten oder Rechtsanwalt. Die Datenbankseite liefert die Umsetzbarkeit.

13. Betrieb und Schema-Evolution

STUFE 3 BIS 4

13.1 Online-DDL in InnoDB

Ein `ALTER TABLE` auf einer Tabelle mit 200 Millionen Zeilen kann Stunden dauern. Entscheidend ist, ob die Tabelle währenddessen lesbar und schreibbar bleibt. InnoDB bietet drei Algorithmen:

ALGORITHMUS	VERHALTEN	TYPISCHE OPERATIONEN
INSTANT	Nur Metadaten, Millisekunden, kein Kopieren	Spalte am Ende hinzufügen (ab 10.3 auch an beliebiger Position), Spalte löschen (10.4+), Standardwert ändern, VARCHAR verlängern (innerhalb Längensklasse), Spalte umbenennen
INPLACE	Tabelle bleibt lesbar und meist schreibbar, Änderungen werden nachgeholt	Index anlegen und löschen, Fremdschlüssel hinzufügen, NULL zu NOT NULL
COPY	Tabelle wird vollständig kopiert, während der Kopie nur lesbar	Datentyp ändern, Zeichensatz ändern, Primärschlüssel ändern

```
ALTER TABLE mietvertrag ADD COLUMN kautio DECIMAL(10,2) NULL, ALGORITHM=INSTANT;  
ALTER TABLE mietvertrag ADD INDEX ix_mv_kautio (kautio), ALGORITHM=INPLACE, LOCK=NONE;  
-- Wenn der gewünschte Algorithmus nicht möglich ist, schlägt das Statement fehl statt  
-- still zu kopieren.
```

Auch ein `INPLACE-ALTER` benötigt am Anfang und Ende kurz einen exklusiven Metadata Lock. Läuft in diesem Moment eine lange Transaktion auf der Tabelle, wartet das `ALTER`, und alle nachfolgenden Abfragen warten hinter dem `ALTER`. Deshalb: `SET SESSION lock_wait_timeout = 5` vor jedem DDL, und bei Fehlschlag später erneut versuchen. Für `COPY`-Operationen auf großen Tabellen im Dauerbetrieb: `pt-online-schema-change` oder `gh-ost`, die eine Schattentabelle aufbauen und per Trigger bzw. Binlog synchron halten.

13.2 Migrationen als Code

Schemaänderungen gehören versioniert ins Repository, werden in Entwicklung, Test und Produktion in derselben Reihenfolge angewendet und sind idempotent oder mit Versionstabelle abgesichert. Werkzeuge: Flyway, Liquibase, oder die Migrationsmechanismen des jeweiligen Frameworks. Jede Migration hat einen Rückwärtspfad oder ist explizit als nicht umkehrbar gekennzeichnet. Destruktive Änderungen (Spalte löschen) erfolgen in zwei Deployments: erst Code, der die Spalte nicht mehr nutzt, dann Löschung.

13.3 Upgrades

MariaDB veröffentlicht Long-Term-Support-Versionen (10.6, 10.11, 11.4, 11.8) mit fünf Jahren Wartung und kurzlebige Rolling Releases. Für den Betrieb ausschließlich LTS verwenden. Upgrade-Pfad: Release Notes und Deprecations lesen, Backup, auf Testsystem mit Produktivdaten upgraden und Anwendung testen, dann Replikat zuerst upgraden, Verhalten beobachten, umschalten, alten Primary upgraden. Nach jedem Upgrade `mariadb-upgrade` ausführen, um Systemtabellen anzupassen. Versionssprünge über mehr als eine Hauptversion sind möglich, Downgrades im Allgemeinen nicht.

13.4 Wiederkehrende Wartung

AUFGABE	RHYTHMUS	HINWEIS
Backup-Wiederherstellung testen	Monatlich	Auf isoliertem System, Dauer messen
Slow Log auswerten	Wöchentlich	Top 10 Muster, Trend beobachten
Ungenutzte Indizes prüfen	Quartalsweise	Nur bei laufendem Performance Schema aussagekräftig
Statistiken aktualisieren	Nach Massenänderungen	ANALYZE TABLE ... PERSISTENT FOR ALL
Tabellenwachstum	Monatlich	information_schema.TABLES, Fragmentierung DATA_FREE
Sicherheitsupdates	Nach Erscheinen	Minor-Releases innerhalb einer LTS-Linie sind risikoarm
Passwortrotation, Rechte-Review	Halbjährlich	Verwaiste Konten entfernen
Kapazitätsplanung	Quartalsweise	Wachstum Daten, Buffer Pool Trefferquote, I/O-Auslastung
Error Log prüfen	Automatisiert täglich	Warnungen zu Aborted Connections, InnoDB, Replikation

13.5 Datenimport in großem Stil

```
-- Schnellster Weg für Massendaten
SET SESSION foreign_key_checks = 0, unique_checks = 0, sql_log_bin = 0; -- nur wenn vertretbar
LOAD DATA LOCAL INFILE '/pfad/daten.csv' INTO TABLE ziel
  FIELDS TERMINATED BY ';' ENCLOSED BY '"' LINES TERMINATED BY '\n' IGNORE 1 LINES
  (spalte1, spalte2, @roh) SET spalte3 = STR_TO_DATE(@roh, '%d.%m.%Y');
SET SESSION foreign_key_checks = 1, unique_checks = 1, sql_log_bin = 1;
```

Weitere Beschleuniger: Sekundärindizes erst nach dem Laden anlegen, Daten nach Primärschlüssel vorsortieren, in einer Transaktion je 10.000 bis 100.000 Zeilen committen, innodb_flush_log_at_trx_commit = 2 während des Imports.

13.6 Container und Cloud

MariaDB läuft problemlos in Docker und Kubernetes (offizielle Images, MariaDB Operator). Beachten: persistente Volumes mit guter I/O-Leistung, Speicherlimit des Containers oberhalb Buffer Pool plus Verbindungspuffer, Health Checks, kein automatisches Löschen von Volumes. Managed-Angebote (AWS RDS for MariaDB, Azure Database for MariaDB, SkySQL) nehmen Backup, Patching und Failover ab, kosten dafür Kontrolle über Parameter und Erweiterungen und einen Aufpreis. Für kleine bis mittlere Systeme ohne eigenen Datenbankbetrieb ist Managed meist die wirtschaftlich richtige Wahl; ab mehreren großen Instanzen rechnet sich eigener Betrieb.

14. Fortgeschrittene Funktionen STUFE 4

14.1 JSON

MariaDB speichert JSON als LONGTEXT mit CHECK (JSON_VALID(...)) (anders als MySQL, das ein Binärformat nutzt). Der Vorteil: Flexibilität für selten abgefragte, heterogene Attribute (Ausstattungsmerkmale, Konfiguration). Der Nachteil: keine direkte Indizierung, Parsing bei jedem Zugriff. Werte, nach denen gefiltert oder sortiert wird, gehören in reguläre Spalten oder in indizierte virtuelle Spalten.

```
CREATE TABLE einheit_merkmale (
  einheit_id INT UNSIGNED PRIMARY KEY,
  merkmale JSON NOT NULL,
  balkon TINYINT(1) AS (JSON_VALUE(merkmale, '$.balkon')) VIRTUAL,
  INDEX ix_balkon (balkon)
);
INSERT INTO einheit_merkmale VALUES (1, '{"balkon": true, "etage": 3, "heizung": "Gas"}');
SELECT einheit_id, JSON_VALUE(merkmale, '$.heizung') FROM einheit_merkmale WHERE balkon = 1;
UPDATE einheit_merkmale SET merkmale = JSON_SET(merkmale, '$.heizung', 'Wärmepumpe')
WHERE einheit_id = 1;
SELECT JSON_ARRAYAGG(JSON_OBJECT('id', einheit_id, 'lage', lage)) FROM einheit WHERE objekt_id = 7;
```

14.2 System-Versioned Tables (Temporal Tables)

Ein Alleinstellungsmerkmal von MariaDB (seit 10.3, SQL:2011-Standard): Die Tabelle führt automatisch eine vollständige Änderungshistorie. Jede Zeile trägt einen Gültigkeitszeitraum; UPDATE und DELETE erzeugen historische Versionen statt sie zu überschreiben. Abfragen "wie sah der Datensatz am 31.12.2025 aus" werden trivial. Für Miethistorien, Eigentümerwechsel, Preislisten und alles, was revisionssicher nachvollziehbar sein muss, ist das der eleganteste Mechanismus.

```
CREATE TABLE eigentum (
  einheit_id INT UNSIGNED NOT NULL,
  eigentuemer_id INT UNSIGNED NOT NULL,
  anteil DECIMAL(9,6) NOT NULL,
  PRIMARY KEY (einheit_id, eigentuemer_id)
) WITH SYSTEM VERSIONING;

-- Stand zu einem Zeitpunkt
SELECT * FROM eigentum FOR SYSTEM_TIME AS OF TIMESTAMP '2025-12-31 23:59:59';
-- Alle Versionen einer Einheit
SELECT *, ROW_START, ROW_END FROM eigentum FOR SYSTEM_TIME ALL WHERE einheit_id = 815;
-- Historie partitionieren, um die aktive Tabelle klein zu halten
ALTER TABLE eigentum PARTITION BY SYSTEM_TIME INTERVAL 1 YEAR (PARTITION p0 HISTORY,
PARTITION pcur CURRENT);
```

Ergänzend gibt es Application-Time Periods (PERIOD FOR gueltig(von, bis), seit 10.4) für fachliche Gültigkeitszeiträume mit UPDATE ... FOR PORTION OF, sowie bitemporale Tabellen als Kombination beider.

14.3 Sequences

Seit 10.3 als eigenständige Objekte, unabhängig von Tabellen, mit NEXTVAL, PREVIOUS VALUE, Cache und Zyklus. Nützlich für Nummernkreise (Rechnungsnummern je Gesellschaft), die über mehrere Tabellen hinweg eindeutig sein müssen.

```
CREATE SEQUENCE seq_rechnung_mhag START WITH 2026001 INCREMENT BY 1 NOCACHE;
INSERT INTO rechnung (nummer, ...) VALUES (NEXTVAL(seq_rechnung_mhag), ...);
```

Hinweis zur Lückenlosigkeit: Weder AUTO_INCREMENT noch Sequences garantieren lückenlose Nummern (Rollbacks verbrauchen Werte). Wo das Handelsrecht fortlaufende Nummern verlangt, wird die Nummer erst beim endgültigen Festschreiben aus einer gesperrten Zählerzeile gezogen.

14.4 Volltextsuche

```
ALTER TABLE dokument ADD FULLTEXT INDEX ft_dokument (titel, inhalt);
SELECT titel, MATCH(titel, inhalt) AGAINST('Eigentümerversammlung Beschluss' IN NATURAL
LANGUAGE MODE) AS relevanz
FROM dokument WHERE MATCH(titel, inhalt) AGAINST('Eigentümerversammlung Beschluss')
ORDER BY relevanz DESC LIMIT 20;
SELECT * FROM dokument WHERE MATCH(inhalt) AGAINST('+Heizkosten -Wasser' IN BOOLEAN
MODE);
```

Grenzen: Mindestwortlänge (innodb_ft_min_token_size, Standard 3), keine Stemming-Unterstützung für Deutsch, keine Fehlertoleranz. Für anspruchsvolle Suche (Facetten, Synonyme, Tippfehler) bleiben Elasticsearch, OpenSearch oder Meilisearch die bessere Wahl, gespeist per Change Data Capture aus dem Binlog (Debezium, Maxwell).

14.5 Vektorsuche

MariaDB 11.7 (Anfang 2025) hat den Datentyp VECTOR und Vektorindizes eingeführt (HNSW-Verfahren, Distanzfunktionen VEC_DISTANCE_EUCLIDEAN, VEC_DISTANCE_COSINE), enthalten auch in LTS 11.8. Damit lassen sich Embeddings aus Sprachmodellen direkt neben den Fachdaten speichern und semantisch durchsuchen (ähnliche Dokumente, Klassifikation von E-Mails, Suche in Vertragstexten). Für Anwendungen mittlerer Größe erspart das eine separate Vektordatenbank. Syntax und Leistungsverhalten sind neu und sollten gegen die Dokumentation der eingesetzten Version geprüft werden.

14.6 Weitere MariaDB-Spezifika

FUNKTION	NUTZEN
INSERT ... RETURNING, DELETE ... RETURNING	Erzeugte Werte ohne zweite Abfrage erhalten
SQL_MODE = ORACLE	PL/SQL-Kompatibilität für Migrationen von Oracle
Invisible Columns, Invisible Indexes	Spalten, die SELECT * nicht liefert; Indizes testweise ausblenden, bevor man sie löscht
UUID-Datentyp (10.7)	Kompakte 16-Byte-Speicherung, korrekte Sortierung
INET6-Datentyp	IP-Adressen nativ

Flashback (mariadb-binlog -- flashback)	DML rückwärts anwenden, um versehentliche Änderungen zurückzunehmen (ROW-Binlog, keine DDL)
Table Elimination, Condition Pushdown	Optimizer-Fähigkeiten, die bestimmte Joins und Derived Tables erheblich beschleunigen
Storage-Engine-übergreifende Statistiken	Histogramme unabhängig von der Engine (Kapitel 7)
JSON_TABLE, JSON_DETAILED, JSON_EQUALS	JSON-Verarbeitung im SQL
Zeilenlimit für Speicherverbrauch (max_statement_time)	Abfragen automatisch nach n Sekunden abbrechen, serverweit oder je Benutzer

14.7 Anwendungsseitige Muster

- **Verbindungspooling:** Verbindungsaufbau kostet Millisekunden und einen Thread. Pool in der Anwendung (HikariCP, pgbouncer-Äquivalente wie ProxySQL) mit Größe etwa CPU-Kerne der Datenbank mal zwei.
- **Caching:** Lesezugriffe auf selten geänderte Daten (Stammdaten, Konfiguration) im Anwendungscache oder Redis; Invalidierung bei Änderung. Die schnellste Abfrage ist die, die nicht gestellt wird.
- **Asynchrone Verarbeitung:** Alles, was nicht sofort in der Antwort gebraucht wird (Benachrichtigungen, PDF-Erzeugung, Statistikaktualisierung), in eine Warteschlange. Eine Tabelle als Queue funktioniert mit `SELECT ... FOR UPDATE SKIP LOCKED` (seit 10.6) sauber.
- **Idempotenz:** Schreiboperationen so gestalten, dass Wiederholung nach Fehler keinen Schaden anrichtet (eindeutige Vorgangsnummer mit UNIQUE-Constraint).
- **Retry bei Deadlock und Lock Timeout:** Fehler 1213 und 1205 werden mit kurzer Wartezeit wiederholt, maximal drei Versuche.

15. Einordnung im Vergleich

STUFE 4

15.1 MariaDB und MySQL

ASPEKT	MARIADB	MYSQL
Lizenz und Steuerung	GPLv2, MariaDB Foundation und MariaDB plc; Community und Enterprise weitgehend deckungsgleich	GPLv2 Community, proprietäre Enterprise mit exklusiven Funktionen (Threadpool, Audit, TDE); Oracle
Optimizer	Eigenständig, Hash Join seit 5.3, Histogramme, Condition Pushdown, Table Elimination	Eigenständig, Hash Join seit 8.0.18, Histogramme seit 8.0
Temporal Tables	System- und Application-Time, bitemporal	Nein
Sequences	Ja	Nein
Storage Engines	InnoDB, Aria, MyRocks, ColumnStore, Spider, S3, CONNECT	InnoDB, MyISAM, NDB
JSON	Text mit Funktionen, JSON_TABLE	Binäres JSON, Multi-Valued Indexes, JSON_TABLE
Galera	Integriert	Nicht integriert (Group Replication als Alternative)
Replikations-GTID	Eigenes Format, inkompatibel	UUID-basiert
Kompatibilität	Divergiert seit 10.x, MySQL-8-Funktionen teilweise fehlend oder anders	Ab 8.0 eigene Wege (Data Dictionary, Roles, Window Functions ähnlich)
Wartungsmodell	LTS 5 Jahre, Rolling Releases dazwischen	LTS und Innovation Releases seit 8.1

Fazit: Für neue Projekte im MySQL-Ökosystem ist MariaDB eine gleichwertige, in Teilen (Temporal Tables, Threadpool, Galera) überlegene Wahl mit vollständig offener Lizenz. Wer Software einsetzt, die ausdrücklich MySQL 8 voraussetzt (z. B. wegen JSON-Multi-Valued-Indexes oder bestimmter Syntax), prüft die Kompatibilität vorher.

15.2 MariaDB und PostgreSQL

ASPEKT	MARIADB	POSTGRESQL
Speicherarchitektur	Clustered Index, Undo Log, Buffer Pool	Heap-Tabellen, alte Versionen in der Tabelle (Vacuum nötig), Shared Buffers plus OS Cache
Isolationsstandard	REPEATABLE READ	READ COMMITTED, echte Serializable Snapshot Isolation verfügbar
Erweiterbarkeit	Storage Engines, Plugins	Extensions (PostGIS, TimescaleDB, pgvector), eigene Typen, Operatoren
Indexarten	B+-Baum, Fulltext, Spatial, Vector	B-Baum, Hash, GIN, GiST, BRIN, partielle und funktionale Indizes nativ

Replikation	Binlog-basiert, logisch (ROW) und mehrere Topologien	Physisch (WAL) und logisch, sehr robust; Multi-Primary nur über Erweiterungen
Typische Stärke	Web-OLTP, Lese-Skalierung, einfacher Betrieb, große Verbreitung im PHP/Web-Umfeld	Komplexe Abfragen, Analytik, Standardtreue, Datentypen, Geodaten
Schwächen	Analytische Abfragen auf großen Datenmengen ohne ColumnStore, weniger Indexarten	Vacuum-Verwaltung, Verbindungskosten (Prozess je Verbindung), Replikationslag bei hoher Schreiblast

Beide sind ausgereifte, produktionsstaugliche Systeme. Die Wahl richtet sich nach Ökosystem der Anwendung, vorhandenem Know-how und Anforderungsprofil, nicht nach absoluter Überlegenheit. Wer beide kennt, ist als Datenbankexperte deutlich breiter aufgestellt.

15.3 Wann NoSQL

- Datenmengen und Schreibraten, die einen einzelnen relationalen Server nachweislich überfordern, mit Zugriffsmustern, die ohne Joins auskommen (Ereignisströme, Sensordaten): Cassandra, ScyllaDB, ClickHouse.
- Dokumente mit stark variierender Struktur, die immer als Ganzes gelesen und geschrieben werden: MongoDB. Prüfen, ob JSON-Spalten in MariaDB nicht ausreichen.
- Latenz im Sub-Millisekunden-Bereich für einfache Schlüssel-Wert-Zugriffe: Redis, als Cache vor der relationalen Datenbank.
- Tiefe Beziehungsnetze mit variabler Pfadlänge als Kernabfrage: Graphdatenbank. Rekursive CTEs decken flache Hierarchien in MariaDB ab.

In allen Fällen bleibt die relationale Datenbank meist das System of Record für Stamm- und Buchungsdaten, das NoSQL-System ergänzt für einen spezifischen Zweck.

16. Lernplan, Übungen, Checklisten, Glossar

16.1 Lernplan über zwölf Wochen

WOCHE	SCHWERPUNKT	PRAXIS
1 bis 2	Kapitel 1 bis 3: Modell, SQL vollständig	MariaDB lokal in Docker installieren, Beispieldatenbank (eigenes Verwaltungsschema oder Employees/Sakila) aufsetzen, 50 Abfragen von einfach bis Window Functions schreiben
3	Kapitel 4: Architektur	SHOW ENGINE INNODB STATUS lesen lernen, Buffer Pool bewusst zu klein stellen und Effekt messen
4	Kapitel 5: Transaktionen	Zwei Sitzungen parallel öffnen, jede Anomalie und einen Deadlock gezielt erzeugen, unter READ COMMITTED wiederholen
5 bis 6	Kapitel 6 und 7: Indizes und Optimizer	Tabelle mit 10 Millionen Zeilen erzeugen (Generator-Skript), jede Anti-Muster-Abfrage messen, Index anlegen, erneut messen, EXPLAIN und ANALYZE FORMAT=JSON vergleichen
7	Kapitel 8 und 9: Tuning und Monitoring	sysbench-Lasttest; drei Parameter einzeln ändern und Durchsatz messen; PMM aufsetzen; Slow Log mit pt-query-digest auswerten
8	Kapitel 10: Replikation	Primary und Replikat in Docker, GTID-Replikation einrichten, Replikat absichtlich brechen und reparieren, Failover durchspielen
9	Kapitel 11: Backup	Mariabackup voll und inkrementell, Wiederherstellung, Point-in-Time-Recovery nach simuliertem DROP TABLE
10	Kapitel 12 und 13: Sicherheit und Betrieb	Rollenmodell aufbauen, TLS erzwingen, Audit Plugin aktivieren, Online-ALTER auf großer Tabelle mit paralleler Last, Upgrade 10.11 auf 11.8 durchführen
11	Kapitel 14: Fortgeschrittenes	System-Versioned Table für Eigentumsverhältnisse, JSON-Merkmale mit indizierter virtueller Spalte, Volltext, Vektorsuche mit Embeddings aus einem Sprachmodell
12	Kapitel 15 und Wiederholung	Ein PostgreSQL parallel aufsetzen und dieselben Übungen aus Woche 5 dort ausführen; Unterschiede notieren. Eigenes Betriebshandbuch für ein reales System schreiben.

16.2 Zertifizierung und vertiefende Quellen

- MariaDB Knowledge Base (mariadb.com/kb): Primärquelle, vollständig und aktuell. Bei jeder Unsicherheit dort nachschlagen, nicht in Foren.
- MariaDB plc bietet Schulungen und eine Zertifizierung (MariaDB Certified Database Administrator); Inhalte und Verfügbarkeit vor Buchung prüfen.
- Bücher: "High Performance MySQL" (Bartos, Tinley, 4. Auflage) bleibt trotz MySQL-Fokus das Standardwerk für InnoDB-Internia und Tuning, nahezu vollständig auf MariaDB übertragbar. "SQL Performance Explained" (Markus Winand) erklärt Indexierung datenbankübergreifend auf 200 Seiten besser als jede andere Quelle. "Designing Data-Intensive Applications" (Martin Kleppmann) für Replikation, Konsistenz und verteilte Systeme.
- Blogs: MariaDB Foundation, Percona Database Performance Blog, Planet MySQL, use-the-index-luke.com.
- Werkzeuge zum Lernen: sysbench für Last, `mysqlslap`, Percona Toolkit, PMM.

16.3 Checkliste: Neue Tabelle

- Expliziter, kurzer, monoton wachsender Primärschlüssel
- InnoDB, utf8mb4, sinnvolle Kollation
- Kleinste passende Datentypen, DECIMAL für Geld, echte Datumstypen
- NOT NULL wo fachlich möglich, sinnvolle Defaults
- Fremdschlüssel und CHECK-Constraints für alle Regeln, die der Server prüfen kann
- Indizes für alle bekannten WHERE- und JOIN-Spalten, zusammengesetzt nach der Präfixregel, keine Redundanz
- Kommentare an Tabelle und unklaren Spalten (COMMENT '...')
- Bei Historisierungsbedarf: System Versioning erwägen
- Bei Zeitreihen mit Aufbewahrungsfrist: Partitionierung erwägen

16.4 Checkliste: Neuer Server

- LTS-Version, aktueller Minor-Release
- Buffer Pool 50 bis 70 % RAM, Redo Log passend zur Schreiblast, O_DIRECT, io_capacity zur Hardware
- Threadpool aktiv, max_connections realistisch, Verbindungspooling in der Anwendung
- Binlog ROW mit GTID, auch ohne aktuelles Replikat
- Slow Log an, long_query_time unter einer Sekunde, Performance Schema an
- Strikter sql_mode, utf8mb4 als Serverstandard
- bind_address eingeschränkt, TLS erzwungen, local_infile aus, skip_name_resolve
- Rollenmodell, kein Anwendungszugriff als root, unix_socket für lokale Admins
- Backup mit Mariabackup eingerichtet, Binlog-Sicherung, Wiederherstellung einmal getestet
- Monitoring mit Alarmen: Verfügbarkeit, Replikationslag, Plattenplatz, Threads_running, Buffer Pool Trefferquote, fehlgeschlagene Backups
- OS: swappiness 1, THP aus, noatime, Dateideskriptor-Limit erhöht
- Dokumentation: Zweck, Verantwortliche, Konfigurationsabweichungen, Wiederherstellungsanleitung

16.5 Checkliste: Langsame Abfrage

- Aus dem Slow Log identifiziert, nicht vermutet
- ANALYZE FORMAT=JSON ausgeführt, teuersten Schritt gefunden
- type ALL oder index auf großer Tabelle: Index fehlt
- rows gegen r_rows verglichen: Statistiken aktuell?
- Funktion oder Typumwandlung auf indizierter Spalte?
- Using filesort oder Using temporary bei großer Menge: Index für ORDER BY / GROUP BY
- Join ohne Index (Using join buffer)?
- Covering Index möglich und lohnend?
- Abfrage umformulierbar (Unterabfrage zu Join, OR zu UNION, OFFSET zu Keyset)?
- Muss die Abfrage überhaupt so oft laufen (Cache, N+1)?

16.6 Glossar

BEGRIFF	BEDEUTUNG
ACID	Atomarität, Konsistenz, Isolation, Dauerhaftigkeit: Garantien einer Transaktion
B+-Baum	Balancierte Baumstruktur aller InnoDB-Indizes, Daten nur in verketteten Blättern
Binlog	Binärlog des Servers mit allen Datenänderungen, Basis für Replikation und PITR
Buffer Pool	Hauptspeichercache von InnoDB für Daten- und Indexseiten
Cardinality	Anzahl verschiedener Werte in Spalte oder Index
Checkpoint	Zeitpunkt, bis zu dem alle Redo-Einträge in die Datendateien geschrieben sind
Clustered Index	Primärschlüsselindex, dessen Blätter die vollständigen Zeilen enthalten; die Tabelle selbst
Covering Index	Index, der alle von einer Abfrage benötigten Spalten enthält
DDL / DML	Data Definition Language (CREATE, ALTER) / Data Manipulation Language (INSERT, UPDATE, DELETE, SELECT)
Deadlock	Zyklisches Warten zweier Transaktionen auf Sperren, wird von InnoDB aufgelöst
Dirty Page	Im Buffer Pool geänderte, noch nicht zurückgeschriebene Seite
fsync	Systemaufruf, der Daten physisch auf das Speichermedium schreibt; Latenzbestimmend für Commits
Gap Lock	Sperre einer Lücke zwischen Indexwerten, verhindert Einfügungen (Phantome)
GTID	Global Transaction ID, eindeutige Kennung einer Transaktion über alle Server einer Replikationstopologie
Histogramm	Statistik über die Werteverteilung einer Spalte für den Optimizer
Isolation Level	Grad der Abschottung gleichzeitiger Transaktionen
LSN	Log Sequence Number, fortlaufende Byteposition im Redo Log
MDL	Metadata Lock, schützt Tabellenstruktur während der Nutzung
MVCC	Multi-Version Concurrency Control: Leser sehen Snapshots, blockieren keine Schreiber
OLTP / OLAP	Transaktionale Kleinzugriffe / analytische Großabfragen
Page	16-KB-Einheit der Speicherverwaltung in InnoDB
PITR	Point-in-Time-Recovery, Wiederherstellung auf einen beliebigen Zeitpunkt via Backup plus Binlog
Purge	Hintergrundprozess, der nicht mehr benötigte Undo-Einträge entfernt
Redo Log	Write-Ahead Log von InnoDB für Crash Recovery
Replication Lag	Zeitlicher Rückstand eines Replikats gegenüber dem Primary
RPO / RTO	Maximal akzeptierter Datenverlust / maximal akzeptierte Wiederherstellungsdauer
Selektivität	Anteil verschiedener Werte an allen Zeilen; hoch ist gut für Indizes
Semi-Join	Join, der nur prüft, ob ein Partner existiert (aus IN/EXISTS)
Sharding	Horizontale Verteilung von Daten nach Schlüssel auf mehrere Server
Storage Engine	Austauschbare Speicherschicht unter dem MariaDB-Server

Tablespace	Datei oder Dateigruppe, in der InnoDB Tabellen speichert
Undo Log	Alte Zeilenversionen für Rollback und MVCC
Working Set	Menge der tatsächlich regelmäßig zugegriffenen Daten und Indizes
Writeset	Bündel der Änderungen einer Transaktion, das Galera repliziert

SCHLUSSBEMERKUNG

Datenbankexpertise ist zu 20 Prozent Wissen und zu 80 Prozent Gewohnheit: jede Abfrage mit EXPLAIN prüfen, jede Tabelle mit Bedacht modellieren, jede Konfigurationsänderung messen, jedes Backup testen. Wer diese vier Gewohnheiten ein Jahr lang durchhält, ist im Ergebnis besser als die meisten, die den Titel tragen. Dieses Dokument ist der Ausgangspunkt; die Knowledge Base von MariaDB und Ihre eigene Testinstanz sind der Weg.